

Sumário

Data Structures	1
Dsu	1
Mo	1
Fenwick Tree	1
Fenwick Tree 2D	2
Dynamic Fenwick Tree 2D	2
Sparse Table	2
Disjoint Sparse Table	2
Segment Tree	2
Lazy Segment Tree	3
Dual Segment Tree	3
Segment Tree Beats	3
Dynamic Segment Tree	4
Dynamic Persistent Segment Tree	4
Line Container (CHT)	4
Graphs	5
Dijkstra	5
MST (Kruskal)	5
Euler Path	5
Lca Sparse Table	5
Lca Binary Lifting	5
Tarjan	5
Articulations	6
Bridges	6
2-SAT	6
Hld Commutative	6
Hld Commutative Lazy	7
Hld Non Commutative	7
Hld Non Commutative Lazy	7
Dinic	8
Min Cost Max Flow	8
Bipartite Matching	8
Math	8
Mint	8
Fast Exponentiation	9
Sieve	9
Euler Phi	9
Number Theory	9
Miller Rabin	9
Matrix	9
Gaussian Elimination	10
Linear Basis	10
Unimodal Search	10
FFT	10
FFT Mod	10
NTT	11
FWHT	11
String	11
KMP	11
Z Function	11
String Hash	11
Trie	12
Manacher	12
Suffix Array (NLogN)	12
Suffix Array (Linear)	12
Geometry	13
Point	13
Line	13
Circle	13
Polygon	13
Convex Hull	14
Closest Pair	14
Others	14
Stress Test	14
Gen	14
Checker	14

Hash Function	14
Hash Verifier	14
Reference Tables	15
Types, Constants & Complexity	15
Combinatória	15
Somatórios	15
Identidades Binomiais	15
Contagem	16
Sequências Especiais	16
Funções Geradoras	16
Potências Fatoriais & Bernoulli	16
Teoria dos Números	16
Euler Phi & Möbius	16
GCD, Diofantinas & TCR	16
Aritmetica Modular	16
Recorrências & Séries	16
Teorema Mestre	16
Séries de Taylor	16
Teoria dos Grafos	16
Teoremas	16
Tipos de Grafo	16
Matemática	16
Probabilidade	16
Trigonometria	17
Geometria	17
Teoria dos Jogos	17
Tabelas Numéricas	17
Números Primos	17
Fibonacci	17
Triângulo de Pascal $\binom{n}{k}$	17
Debugging	17

Data Structures

Dsu

```

/* DSU (Disjoint Set Union / Union-Find)
 * Mantem conjuntos disjuntos com uniao e busca eficiente.
 * Complexidade: O(a(N)) por operacao, onde a e a inversa de Ackermann.
 * Memoria: O(N)
 * Metodos:
 * - find(i): Retorna o representante do conjunto de i.
 * - join(i, j): Une os conjuntos de i e j, retorna true se uniu.
 * - same(i, j): Verifica se i e j estao no mesmo conjunto.
 * - size(i): Retorna o tamanho do conjunto de i.
 */

```

```

D5E struct DSU {
1A8     int n;
923     vector<int> parent;
1DF     vector<int> sz;
75E     int num_sets;
DDB     DSU(int _n) : n(_n), parent(_n), sz(_n, 1), num_sets(_n) {
4D6         iota(parent.begin(), parent.end(), 0);
548     }
C42     int find(int i) {
331         if (parent[i] == i) return i;
565         return parent[i] = find(parent[i]);
72C     }
D58     bool join(int i, int j) {
477         i = find(i), j = find(j);
41E         if (i != j) {
4C2             if (sz[i] < sz[j]) swap(i, j);
1F0             parent[j] = i;
529             sz[i] += sz[j];
CA8             num_sets--;
8A6             return true;
20F         }
D1F         return false;
30C     }
00C     int size(int i) { return sz[find(i)]; }
DA3     bool same(int i, int j) { return find(i) == find(j); }
FF6 };

```

Mo

```

/* Mo's Algorithm - O((N + Q) * sqrt(N))
 * queries: vetor de {l, r} 0-indexado, fechado.
 * add(i), rem(i): adicionam/removem o elemento i da janela.
 * ans(qi): registra resposta para a query de indice qi.
 *

```

```

* Exemplo:
*   vector<int> ans(q);
*   Mo mo(n, queries);
*   mo.run(
*       [&](int i) { ... },
*       [&](int i) { ... },
*       [&](int qi) { ans[qi] = ...; }
*   );
*/
D0F struct Mo {
7F4     int n, block;
304     struct Query { int l, r, idx; };
240     vector<Query> queries;
8CA     Mo(int n, vector<pair<int,int>&> &qs)
10A         : n(n), block(max(1, (int)sqrt(n))) {
D26         for (int i = 0; i < (int)qs.size(); i++)
9AA             queries.push_back({qs[i].first, qs[i].second, i});
F94         sort(queries.begin(), queries.end(),
724             [&](const Query& a, const Query& b) {
ED6             int ba = a.l / block, bb = b.l / block;
A2D             if (ba != bb) return ba < bb;
FDF             return (ba & 1) ? a.r > b.r : a.r < b.r;
FC3         });
8F7     }
D5A     template<typename Add, typename Rem, typename Ans>
743     void run(Add add, Rem rem, Ans ans) {
70A         int L = 0, R = -1;
FFE         for (auto& [l, r, idx] : queries) {
DBE             while (R < r) add(++R);
EBF             while (L > l) add(--L);
153             while (R > r) rem(R--);
319             while (L < l) rem(L++);
414             ans[idx];
4B3         }
1D9     }
09E };

```

```

/* HilbertMo - Mo com ordenacao pela curva de Hilbert
 * Mesma interface que Mo, mas com constante menor na pratica.
 */

```

```

CD0 struct HilbertMo {
22A     struct Query { int l, r, idx; long long ord; };
240     vector<Query> queries;
B0A     static long long hilbert(int x, int y, int n) {
CEE         long long d = 0;
F8A         for (int s = n >> 1; s >= 1; s >>= 1) {
7B9             int rx = (x & s) > 0, ry = (y & s) > 0;
065             d += (long long)s * s * ((3 * rx) ^ ry);
069             if (!ry) {
1FD                 if (rx) { x = s - 1 - x; y = s - 1 - y; }
9DD                 swap(x, y);
FFF             }
070         }
BE2         return d;
151     }
E76     HilbertMo(int n, vector<pair<int,int>&> &qs) {
ABE         int hn = 1;
FCC         while (hn < n) hn <= 1;
D26         for (int i = 0; i < (int)qs.size(); i++)
F80             queries.push_back({qs[i].first, qs[i].second, i,
C04                 hilbert(qs[i].first, qs[i].second, hn)});
F94         sort(queries.begin(), queries.end(),
B18             [](const Query& a, const Query& b) {
73A                 return a.ord < b.ord;
A3A             });
0D6     }
D5A     template<typename Add, typename Rem, typename Ans>
743     void run(Add add, Rem rem, Ans ans) {
70A         int L = 0, R = -1;
277         for (auto& [l, r, idx, ord] : queries) {
DBE             while (R < r) add(++R);
EBF             while (L > l) add(--L);
153             while (R > r) rem(R--);
319             while (L < l) rem(L++);
414             ans[idx];
F5F         }
EBB     }
EF4 };

```

Fenwick Tree

```

/* Fenwick Tree (Point Update, Prefix Query)
 * Realiza atualizacoes pontuais e consultas de prefixo.
 * Complexidade: O(log N) para update e query.
 * Memoria: O(N)
 * Requisitos:
 * - NODE deve ter: operator+=, operator- (para range query) e
 *   construtor default.
 */

```

```

/* --- Exemplo de NODE (Soma) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    void operator+=(const Node& other) { val += other.val; }
    Node operator-(const Node& other) const { return Node(val - other.val); }
};
*/

8A0 template<typename NODE>
0F1 struct FenwickTree {
060     int N;
9B8     vector<NODE> bit;

5D2     FenwickTree(int n) : N(n), bit(n + 1) {}

D6B     void update(int idx, NODE v) {
B2F         for (idx++; idx <= N; idx += idx & -idx) {
046             bit[idx] += v;
23E         }
9DA     }

FCA     NODE query(int idx) { // [0, idx]
ADA         NODE res;
159         for (idx++; idx > 0; idx -= idx & -idx) {
8E6             res += bit[idx];
A38         }
B50         return res;
C00     }

762     NODE query(int l, int r) { // [l, r] (precisa ser reversivel)
7C3         if (l > r) return NODE();
A4E         return query(r) - query(l - 1);
821     }
134 };

```

Fenwick Tree 2D

```

/* Fenwick Tree 2D (Point Update, Rectangle Query)
* Realiza atualizacoes pontuais e consultas de prefixo em matrizes.
* Complexidade: O(log N * log M) para update e query.
* Memoria: O(N * M)
* Requisitos:
* - NODE deve ter: operator+==, operator- e construtor default.
*/

/* --- Exemplo de NODE (Soma) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    void operator+=(const Node& other) { val += other.val; }
    Node operator-(const Node& other) const { return Node(val - other.val); }
};
*/

8A0 template<typename NODE>
2E6 struct FenwickTree2D {
610     int N, M;
803     vector<vector<NODE>>> bit;

237     FenwickTree2D(int n, int m) : N(n), M(m), bit(n + 1, vector<NODE>(m + 1)) {}

274     void update(int r, int c, NODE v) { // point update
3BA         for (int i = r + 1; i <= N; i += i & -i) {
786             for (int j = c + 1; j <= M; j += j & -j) {
9ED                 bit[i][j] += v;
95F             }
A1F         }
BF9     }

82E     NODE query(int r, int c) { // [(0,0), (r,c)]
ADA         NODE res;
8D9         for (int i = r + 1; i > 0; i -= i & -i) {
83F             for (int j = c + 1; j > 0; j -= j & -j) {
01D                 res += bit[i][j];
CB5             }
E20         }
B50         return res;
943     }

05F     NODE query(int r1, int c1, int r2, int c2) { // [(r1,c1), (r2,c2)]
5C4         if (r1 > r2 || c1 > c2) return NODE();
B7F         return query(r2, c2) - query(r1 - 1, c2)
794             - query(r2, c1 - 1) + query(r1 - 1, c1 - 1);
0E7     }
FF7 };

```

Dynamic Fenwick Tree 2D

```

/* Fenwick Tree 2D Sparse (Point Update, Rectangle Query)
* Suporta coordenadas ate 10^9 usando compressao interna.
* Complexidade: Build O(P log P), Update/Query O(log^2 P).
* Memoria: O(P log P).
* Requisitos:
* - NODE deve ter: operator+==, operator-, operator+ e construtor default.
* - Offline: Todos os pontos de interesse devem ser passados no build.
*

```

```

* Creditos: Adaptado de TFG (Thiago Ferreira Goncalves)
* Fonte: github.com/tfg50/Competitive-Programming
*/

```

```

/* --- Exemplo de NODE (Soma com Inclusao-Exclusao) ---
struct Node {
    ll val;
    Node(long long v = 0) : val(v) {}
    void operator+=(const Node& other) {
        val += other.val;
    }

    Node operator-(const Node& other) const {
        return Node(val - other.val);
    }

    Node operator+(const Node& other) const {
        return Node(val + other.val);
    }
};
*/

```

```

8A0 template<typename NODE>
222 struct FenwickTree2DSparse {
6C1     vector<int> ord;
803     vector<vector<NODE>>> bit;
E76     vector<vector<int>>> coord;

BEF     FenwickTree2DSparse(vector<pii> pts) {
CD7         sort(pts.begin(), pts.end());
5A8         for (auto [x, y] : pts) {
DC4             if (ord.empty() || x != ord.back()) ord.push_back(x);
ABA         }

261         bit.resize(ord.size() + 1);
3EB         coord.resize(ord.size() + 1);

A0A         sort(pts.begin(), pts.end(), [](const pii& a, const pii& b) {
463             return a.second < b.second;
19C         });

5A8         for (auto [x, y] : pts) {
A1A             for (int i = get_x(x); i < (int)bit.size(); i += i & -i) {
3E1                 if (coord[i].empty() || coord[i].back() != y)
739                     coord[i].push_back(y);
D32             }
BAF         }

BE8         for (int i = 0; i < (int)bit.size(); i++) {
F3E             bit[i].assign(coord[i].size() + 1, NODE());
B33         }
84E     }

F1C     inline int get_x(int x) {
2C8         return upper_bound(ord.begin(), ord.end(), x) - ord.begin();
FF7     }

DD7     inline int get_y(int i, int y) {
190         return upper_bound(coord[i].begin(), coord[i].end(), y)
7A2             - coord[i].begin();
B44     }

5CF     void update(int x, int y, NODE v) {
A1A         for (int i = get_x(x); i < (int)bit.size(); i += i & -i) {
511             for (int j = get_y(i, y); j < (int)bit[i].size(); j += j & -j) {
9ED                 bit[i][j] += v;
66B             }
638         }
0CE     }

84C     NODE query(int x, int y) {
ADA         NODE res;
FAD         for (int i = get_x(x); i > 0; i -= i & -i) {
263             for (int j = get_y(i, y); j > 0; j -= j & -j) {
01D                 res += bit[i][j];
359             }
92F         }
B50         return res;
7C4     }

FCA     NODE query(int x1, int y1, int x2, int y2) {
CD6         return query(x2, y2) - query(x1 - 1, y2)
CAD             - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
944     }
67D };

```

Sparse Table

```

/* Sparse Table (Static RMQ)
* Realiza consultas em range em tempo constante.
* Complexidade: O(N log N) no build, O(1) na query.
* Memoria: O(N log N)
* Requisitos:
* - NODE deve ter: static merge(const NODE&, const NODE&) e construtor.
* - A operacao DEVE ser idempotente: f(a, a) = a.
*/

```

```

/* --- Exemplo de NODE (Indice do Minimo) ---
struct Node {
    int val, pos;

```

```

Node(int v = 2e9, int p = -1) : val(v), pos(p) {}

static inline Node merge(const Node& l, const Node& r) {
    return l.val <= r.val ? l : r;
}

};

// Uso: SparseTable<Node> st(v) onde v e vector<Node>
// for (int i = 0; i < n; i++) { cin >> x; v[i] = Node(x, i); }
*/

8A0 template<typename NODE>
7E9 struct SparseTable {
5F0     int N, K;
6F5     vector<vector<NODE>>> st;
4A7     vector<int> lg;

67A     template<typename T>
E4A     SparseTable(const vector<T& v) : N(v.size()) {
A77         K = (N > 0) ? __lg(N) : 0;
3A9         st.assign(K + 1, vector<NODE>(N));
641         lg.assign(N + 1, 0);
BB5         for (int i = 2; i <= N; i++) lg[i] = lg[i / 2] + 1;
DDD         for (int i = 0; i < N; i++) st[0][i] = NODE(v[i]);
2CE         for (int j = 1; j <= K; j++)
285             for (int i = 0; i + (1 << j) <= N; i++)
4E1                 st[j][i] = NODE::merge(st[j - 1][i],
A64                     st[j - 1][i + (1 << (j - 1))]);
CE2     }

762     NODE query(int l, int r) {
2C3         int j = lg[r - l + 1];
8F2         return NODE::merge(st[j][l], st[j][r - (1 << j) + 1]);
843     }
231 };

```

Disjoint Sparse Table

```

/* Disjoint Sparse Table (Static Range Query)
* Realiza consultas em range em O(1) para qualquer operacao associativa.
* Complexidade: O(N log N) no build, O(1) na query.
* Memoria: O(N log N)
* Requisitos:
* - NODE deve ter: static merge(const NODE&, const NODE&) e construtor.
* - Operacao deve ser ASSOCIATIVA: f(f(a,b),c) = f(a,f(b,c)).
*/

```

```

/* --- Exemplo de NODE (Soma em Range) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }
};
*/

```

```

8A0 template<typename NODE>
906 struct DisjointSparseTable {
5F0     int N, K;
6F5     vector<vector<NODE>>> st;

67A     template<typename T>
68E     DisjointSparseTable(const vector<T& v) : N(v.size()) {
3AA         if (N == 0) return;
475         K = __lg(2 * N - 1);
C78         st.assign(K, vector<NODE>(1 << K));
DDD         for (int i = 0; i < N; i++) st[0][i] = NODE(v[i]);
C54         for (int j = 1; j < K; j++) {
1C0             int len = 1 << j;
C2C             for (int m = len; m <= (1 << K); m += 2 * len) {
A34                 if (m < N) st[j][m] = st[0][m];
6D6                 for (int i = m + 1; i < min(N, m + len); i++)
0EA                     st[j][i] = NODE::merge(st[j][i - 1], st[0][i]);
C2E                 if (m - 1 < N) st[j][m - 1] = st[0][m - 1];
226                 for (int i = m - 2; i >= m - len; i--)
604                     st[j][i] = NODE::merge(st[0][i], st[j][i + 1]);
643             }
3F0         }
4D7     }

1F9     inline NODE query(int l, int r) {
434         if (l == r) return st[0][l];
B89         int j = __lg(l ^ r);
894         return NODE::merge(st[j][l], st[j][r]);
DB5     }
4D6 };

```

Segment Tree

```

/* Segment Tree (Point Update, Range Query)
* Complexidade: O(log N) para update e query.
* Memoria: O(4*N)
* Requisitos:
* - NODE deve ter: static merge(L, R), apply(V), e construtor identidade.

```

```

*/
/* --- Exemplo de NODE (Soma com Update de Substituicao) ---
struct Node {
    ll val = 0;
    Node(ll v = 0) : val(v) {}

    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }

    // Para Point Set: val = v;
    // Para Point Add: val += v;
    inline void apply(ll v) {
        val = v;
    }
};
*/

8A0 template<typename NODE>
383 struct SegTree {
060     int N;
B1C     vector<NODE> seg;

FDE     explicit SegTree(int n) : N(n), seg(4 * n) {}

67A     template<typename T>
5DE     SegTree(const vector<T>& v) : SegTree((int)v.size()) {
231         build(1, 0, N - 1, v);
73D     }

67A     template<typename T>
5B9     void build(int no, int l, int r, const vector<T>& v) {
893         if (l == r) {
C3E             seg[no] = NODE(v[l]);
505             return;
B2E         }
27E         int m = (l + r) >> 1;
35E         build(no << 1, l, m, v);
C55         build((no << 1) | 1, m + 1, r, v);
DEB         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
349     }

637     template<typename V>
AF7     void update(int no, int l, int r, int idx, V val) {
893         if (l == r) {
D88             seg[no].apply(val);
505             return;
EC9         }
27E         int m = (l + r) >> 1;
B73         if (idx <= m) update(no << 1, l, m, idx, val);
99C         else update((no << 1) | 1, m + 1, r, idx, val);
DEB         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
7BC     }

38D     NODE query(int no, int l, int r, int a, int b) {
2E6         if (b < l || r < a) return NODE();
83F         if (a <= l && r <= b) return seg[no];
27E         int m = (l + r) >> 1;
AFE         return NODE::merge(query(no << 1, l, m, a, b),
074             query((no << 1) | 1, m + 1, r, a, b));
365     }

637     template<typename V>
22F     void update(int idx, V val) { update(1, 0, N - 1, idx, val); }
36F     NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
0C7 };

```

Lazy Segment Tree

```

/* Lazy Segment Tree (Range Update, Range Query)
* Realiza atualizacoes e consultas em range.
* Complexidade: O(log N) para update e query.
* Memoria: O(4*N)
* Requisitos:
* - NODE deve ter: static merge(L, R), apply(TAG, L, R) e
*   construtor identidade.
* - TAG deve ter: compose(TAG) e construtor identidade.
*/

/* --- Exemplo de NODE e TAG (Soma e Multiplicacao com Modulo) ---
struct Tag {
    ll mul = 1, add = 0;
    void inline compose(const Tag& t) {
        add = (add * t.mul + t.add) % MOD;
        mul = (mul * t.mul) % MOD;
    }
};

struct Node {
    ll val = 0;
    Node(ll v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node((l.val + r.val) % MOD);
    }
    void inline apply(const Tag& t, int l, int r) {
        val = (val * t.mul + t.add * (r - l + 1)) % MOD;
    }
};

```

```

};
*/

708 template<typename NODE, typename TAG>
28A struct LazySegmentTree {
060     int N;
B1C     vector<NODE> seg;
71A     vector<TAG> lazy;

9F5     explicit LazySegmentTree(int n) : N(n), seg(4 * n), lazy(4 * n) {}

67A     template<typename T>
52C     LazySegmentTree(const vector<T>& v) : LazySegmentTree((int)v.size()) {
231         build(1, 0, N - 1, v);
E6C     }

67A     template<typename T>
5B9     void build(int no, int l, int r, const vector<T>& v) {
893         if (l == r) {
C3E             seg[no] = NODE(v[l]);
505             return;
B2E         }
27E         int m = (l + r) >> 1;
35E         build(no << 1, l, m, v);
C55         build((no << 1) | 1, m + 1, r, v);
DEB         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
349     }

F08     void push(int no, int l, int r, const vector<T>& v) {
27E         int m = (l + r) >> 1;
1EA         int e = no << 1, d = e | 1;
25F         seg[e].apply(lazy[no], l, m);
115         lazy[e].compose(lazy[no]);

4E1         seg[d].apply(lazy[no], m + 1, r);
7CC         lazy[d].compose(lazy[no]);
47F         lazy[no] = TAG();
6A3     }

130     void update(int no, int l, int r, int a, int b, const TAG& v) {
2E6         if (b < l || r < a) return;
02B         if (a <= l && r <= b) {
8C7             seg[no].apply(v, l, r);
92C             lazy[no].compose(v);
505             return;
81F         }
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
9AB         update(no << 1, l, m, a, b, v);
087         update((no << 1) | 1, m + 1, r, a, b, v);
DEB         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
A1A     }

38D     NODE query(int no, int l, int r, int a, int b) {
2E6         if (b < l || r < a) return NODE();
83F         if (a <= l && r <= b) return seg[no];
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
AFE         return NODE::merge(query(no << 1, l, m, a, b),
074             query((no << 1) | 1, m + 1, r, a, b));
80A     }

0D7     void update(int l, int r, const TAG& v) { update(1, 0, N - 1, l, r, v); }
36F     NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
364 };

```

Dual Segment Tree

```

/* Dual Segment Tree (Range Update, Point Query)
* Realiza atualizacoes em range e consultas pontuais.
* Complexidade: O(log N) para update e get.
* Memoria: O(4*N)
* Requisitos:
* - TAG deve ter: compose(TAG), apply(T& e construtor identidade.
* - T e o tipo do dado nas folhas.
* Exemplo de uso: DualSegTree<ll, MyTag> st(v);
*/

/* --- Exemplo de TAG (Soma e Multiplicacao com Modulo) ---
struct Tag {
    ll mul = 1, add = 0;
    void compose(const Tag& t) {
        mul = (mul * t.mul) % MOD;
        add = (add * t.mul + t.add) % MOD;
    }
    void apply(ll& val) { val = (val * mul + add) % MOD; }
};

2E5 template<typename T, typename TAG>
9C1 struct DualSegTree {
060     int N;
71A     vector<TAG> lazy;
E4B     vector<T> leaves;

F06     explicit DualSegTree(int n) : N(n), lazy(4 * n), leaves(n) {}
5F6     DualSegTree(const vector<T>& v)

```

```

CE6         : N((int)v.size()), lazy(4 * v.size()), leaves(v) {}
F08     void push(int no, int l, int r) {
F7F         int e = no << 1;
EE1         int d = e | 1;
115         lazy[e].compose(lazy[no]);
7CC         lazy[d].compose(lazy[no]);
47F         lazy[no] = TAG();
48C     }

130     void update(int no, int l, int r, int a, int b, const TAG& v) {
2E6         if (b < l || r < a) return;
02B         if (a <= l && r <= b) {
92C             lazy[no].compose(v);
505             return;
CA0         }
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
9AB         update(no << 1, l, m, a, b, v);
087         update((no << 1) | 1, m + 1, r, a, b, v);
667     }

629     T get(int no, int l, int r, int idx) {
893         if (l == r) {
4C5             T res = leaves[idx];
2E3             lazy[no].apply(res);
B50             return res;
BE7         }
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
52E         if (idx <= m) return get(no << 1, l, m, idx);
483         else return get((no << 1) | 1, m + 1, r, idx);
A24     }

A21     void update(int l, int r, const TAG& t) { update(1, 0, N - 1, l, r, t); }
AE4     T get(int idx) { return get(1, 0, N - 1, idx); }
796 };

```

Segment Tree Beats

```

/* Segment Tree Beats (Range Update, Range Query)
* Extensao da Lazy Seg Tree com condicoes de parada e de tag.
* Complexidade amortizada: O(N log^2 N) para sequencias de updates.
* Memoria: O(4*N)
* Requisitos:
* - NODE deve ter: static merge(L, R), apply(TAG, L, R),
*   break_condition(TAG) e tag_condition(TAG),
*   e construtor identidade.
* - TAG deve ter: compose(TAG) e construtor identidade.
*
* break_condition(tag): retorna true se o update pode ser ignorado
*   (o no ja satisfaz a condicao - ex: mx <= v).
* tag_condition(tag): retorna true se a tag pode ser propagada
*   diretamente sem descer (ex: mx2 < v <= mx).
*/

```

```

/* --- Exemplo de NODE e TAG (chmin em range + query de soma/max) ---
struct Tag {
    ll val = LLONG_MAX; // identidade: nao faz nada
    void inline compose(const Tag& t) {
        val = min(val, t.val);
    }
};

struct Node {
    ll sum, mx, mx2;
    int cnt_mx, sz;

    Node() : sum(0), mx(LLONG_MIN), mx2(LLONG_MIN), cnt_mx(0), sz(0) {}
    Node(ll v) : sum(v), mx(v), mx2(LLONG_MIN), cnt_mx(1), sz(1) {}

    static inline Node merge(const Node& l, const Node& r) {
        Node res;
        res.sz = l.sz + r.sz;
        res.sum = l.sum + r.sum;
        if (l.mx == r.mx) {
            res.mx = l.mx; res.cnt_mx = l.cnt_mx + r.cnt_mx;
            res.mx2 = max(l.mx2, r.mx2);
        } else if (l.mx > r.mx) {
            res.mx = l.mx; res.cnt_mx = l.cnt_mx;
            res.mx2 = max(l.mx2, r.mx);
        } else {
            res.mx = r.mx; res.cnt_mx = r.cnt_mx;
            res.mx2 = max(l.mx, r.mx2);
        }
        return res;
    }

    // aplica a tag quando tag_condition e verdadeiro
    void inline apply(const Tag& t, int l, int r) {
        sum -= (ll)(mx - t.val) * cnt_mx;
        mx = t.val;
    }

    // ignora o update completamente (no ja satisfaz)
    bool inline break_condition(const Tag& t) const {
        return mx <= t.val;
    }
};

```

```

}
// pode aplicar a tag diretamente sem descer
bool inline tag_condition(const Tag& t) const {
    return mx2 < t.val;
}
};
*/

708 template<typename NODE, typename TAG>
BC8 struct SegTreeBeats {
060     int N;
B1C     vector<NODE> seg;
71A     vector<TAG> lazy;
249     explicit SegTreeBeats(int n) : N(n), seg(4 * n), lazy(4 * n) {}
67A     template<typename T>
2F9     SegTreeBeats(const vector<T>& v) : SegTreeBeats((int)v.size()) {
231         build(1, 0, N - 1, v);
BBC     }
67A     template<typename T>
5B9     void build(int no, int l, int r, const vector<T>& v) {
893         if (l == r) {
C3E             seg[no] = NODE(v[l]);
505             return;
B2E         }
27E         int m = (l + r) >> 1;
35E         build(no << 1, l, m, v);
C55         build((no << 1) | 1, m + 1, r, v);
DEB         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
349     }
F08     void push(int no, int l, int r) {
27E         int m = (l + r) >> 1;
1EA         int e = no << 1, d = e | 1;
25F         seg[e].apply(lazy[no], l, m);
115         lazy[e].compose(lazy[no]);
4E1         seg[d].apply(lazy[no], m + 1, r);
7CC         lazy[d].compose(lazy[no]);
47F         lazy[no] = TAG();
6A3     }
130     void update(int no, int l, int r, int a, int b, const TAG& v) {
FBB         if (b < l || r < a || seg[no].break_condition(v)) return;
40D         if (a <= l && r <= b && seg[no].tag_condition(v)) {
8C7             seg[no].apply(v, l, r);
92C             lazy[no].compose(v);
505             return;
2B2         }
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
9AB         update(no << 1, l, m, a, b, v);
087         update((no << 1) | 1, m + 1, r, a, b, v);
DEB         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
18B     }
38D     NODE query(int no, int l, int r, int a, int b) {
2E6         if (b < l || r < a) return NODE();
83F         if (a <= l && r <= b) return seg[no];
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
AFE         return NODE::merge(query(no << 1, l, m, a, b),
074             query((no << 1) | 1, m + 1, r, a, b));
80A     }
0D7     void update(int l, int r, const TAG& v) { update(1, 0, N - 1, l, r, v); }
36F     NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
821 };

```

Dynamic Segment Tree

```

/* Dynamic Segment Tree with Lazy Propagation (Range Update, Range Query)
 * Utilizada para intervalos gigantes (ex: 0 a 10^18).
 * Complexidade: O(log N) por operacao.
 * Memoria: O(Q log N) onde Q e o numero de operacoes.
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&),
 *   apply(const TAG&, ll, ll) e construtor identidade.
 * - TAG deve ter: apply(const TAG&), campo 'has' (bool) e
 *   construtor identidade.
 */
/* --- Exemplo de NODE e TAG ---
struct Tag {
    ll add = 0;
    bool has = false;
    void apply(const Tag& t) {
        add += t.add;
        has = true;
    }
};

struct Node {
    ll val = 0;

```

```

static Node merge(const Node& l, const Node& r) {
    return {l.val + r.val};
}

void apply(const Tag& t, ll l, ll r) {
    val += t.add * (r - l + 1);
}

};
*/

708 template<typename NODE, typename TAG>
77D struct DynamicSegTree {
126     struct InternalNode {
4EB         NODE node;
657         TAG tag;
74F         int l, r;
FB6         InternalNode() : node(NODE()), tag(TAG()), l(-1), r(-1) {}
A61         void apply(const TAG& t, ll l, ll r) {
BE6             node.apply(t, l, r);
D21             tag.apply(t);
9C3         }
2FB     };
9B3     ll L, R;
4C1     vector<InternalNode> seg;
5C7     DynamicSegTree(ll l, ll r) : L(l), R(r) {
3BD         seg.reserve(4e6); seg.emplace_back();
BFD     }
4F9     void push(int v, ll l, ll r) {
A16         if (!seg[v].tag.has) return;
56B         if (l == r) return seg[v].tag = TAG();
579         if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
203         if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
769         ll mid = l + (r - l) / 2;
F3E         seg[seg[v].l].apply(seg[v].tag, l, mid);
0B7         seg[seg[v].r].apply(seg[v].tag, mid+1, r);
DD0         seg[v].tag = TAG();
AD7     }
352     void update(int v, ll l, ll r, ll ql, ll qr, const TAG& t) {
151         if (ql <= l && r <= qr) return seg[v].apply(t, l, r);
A48         push(v, l, r);
769         ll mid = l + (r - l) / 2;
441         if (ql <= mid) {
579             if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
2DD             update(seg[v].l, l, mid, ql, qr, t);
0B1         }
64D         if (qr > mid) {
203             if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
76B             update(seg[v].r, mid + 1, r, ql, qr, t);
216         }
C5A         NODE left = (seg[v].l != -1) ? seg[seg[v].l].node : NODE();
C56         NODE rgt = (seg[v].r != -1) ? seg[seg[v].r].node : NODE();
799         seg[v].node = NODE::merge(left, rgt);
15E     }
EE0     NODE query(int v, ll l, ll r, ll ql, ll qr) {
5CF         if (v == -1 || ql > r || qr < l) return NODE();
3C9         if (ql <= l && r <= qr) return seg[v].node;
A48         push(v, l, r);
769         ll mid = l + (r - l) / 2;
211         return NODE::merge(query(seg[v].l, l, mid, ql, qr),
0CC             query(seg[v].r, mid + 1, r, ql, qr));
933     }
4CA     void update(ll ql, ll qr, const TAG& t) { update(0, L, R, ql, qr, t); }
323     NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
4CA };

```

Dynamic Persistent Segment Tree

```

/* Persistent Dynamic Segment Tree (Point Update, Range Query)
 * Permite consultas em versoes anteriores da arvore (persistencia).
 * Aloca nos sob demanda, ideal para intervalos grandes e multiplas versoes.
 * Complexidade: O(log N) por update e query.
 * Memoria: O(Q log N) onde Q e o numero de updates.
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&) e
 *   construtor identidade.
 */
/* --- Exemplo de NODE (Soma) ---
struct Node {
    int val;
    Node(int v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }
};
*/

8A0 template<typename NODE>
68A struct PersistentSegmentTree {

```

```

126 struct InternalNode {
1DF     NODE data;
74F     int l, r;
E61     InternalNode() : data(NODE()), l(0), r(0) {}
B12 };
060
060 int N;
788 vector<InternalNode> st;
34F vector<int> roots;
BC9
BC9 PersistentSegmentTree(int n) : N(n) {
967     st.reserve(4e6); st.emplace_back();
405     roots.push_back(0);
71E }
C93
int update(int prev_root, int L, int R, int idx, NODE v) {
4C0     int node = st.size();
6F5     if (prev_root == 0) st.emplace_back();
FFC     else st.push_back({st[prev_root]});
651     if (L == R) {
DC5         st[node].data = NODE::merge(st[node].data, v);
192         return node;
BE8     }
08E     int mid = L + (R - L) / 2;
D33     if (idx <= mid) { // Cuidado com ordem de avaliacao (LHS) pre-C++17
9DD         int prev_left = (prev_root == 0) ? 0 : st[prev_root].l;
5A1         st[node].l = update(prev_left, L, mid, idx, v);
0CC     }
4E6     else { // Cuidado com ordem de avaliacao (LHS) pre-C++17
709         int prev_right = (prev_root == 0) ? 0 : st[prev_root].r;
C59         st[node].r = update(prev_right, mid + 1, R, idx, v);
777         st[node].data = NODE::merge(st[st[node].l].data, st[st[node].r].data);
192         return node;
F04     }
082     NODE query(int node, int L, int R, int i, int j) {
088         if (node == 0 || i > R || j < L) return NODE();
692         if (i <= L && R <= j) return st[node].data;
08E         int mid = L + (R - L) / 2;
B1E         return NODE::merge(query(st[node].l, L, mid, i, j),
521             query(st[node].r, mid + 1, R, i, j));
26E     }
D6B     void update(int idx, NODE v) {
69E         roots.push_back(update(roots.back(), 0, N - 1, idx, v));
1BF     }
3A5     NODE query(int version, int l, int r) {
191         return query(roots[version], 0, N - 1, l, r);
247     }
B19 };

```

Line Container (CHT)

```

/* Convex Hull Trick dinamico (LineContainer) - O(log N) por operacao
 *
 * Mantem um conjunto de retas y = k*x + m e responde o MAXIMO em um x.
 * Insere e consulta em qualquer ordem - k e x arbitrarios, sem precisar
 * de monotonicidade.
 *
 * Quando usar: DP do tipo dp[i] = max_j (k[j]*x[i] + m[j]) - cada estado vira
 * uma reta add(k[j], m[j]) e a transicao e uma query(x[i]).
 *
 * add(k, m) -> insere a reta y = k*x + m
 * query(x) -> maior valor entre todas as retas em x
 *
 * Para MINIMO: insira add(-k, -m) e use -query(x).
 */
72C struct Line {
525     ll k, m; mutable ll p;
CA5     bool operator<(const Line& o) const { return k < o.k; }
ABF     bool operator<(ll x) const { return p < x; }
868 };
781 struct LineContainer : multiset<Line, less<>> {
FD2     static const ll inf = LLONG_MAX;
33A     ll div(ll a, ll b) {
353         return a / b - ((a ^ b) < 0 && a % b);
10F     }
A1C     bool isect(iterator x, iterator y) {
A95         if (y == end()) return x->p = inf, 0;
9CB         if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
591         else x->p = div(y->m - x->m, x->k - y->k);
870         return x->p >= y->p;
2FA     }
A0C     void add(ll k, ll m) {
116         auto z = insert({k, m, 0}), y = z++, x = y;
7B1         while (isect(y, z)) z = erase(z);
141         if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
57D         while ((y = x) != begin() && (--x)->p >= y->p)
774             isect(x, erase(y));
886     }

```

```

4AD   ll query(ll x) {
229       assert(!empty());
7D1       auto l = *lower_bound(x);
96A       return l.k * x + l.m;
D21   }
577 };

```

Graphs

Dijkstra

```

/* Dijkstra – Caminho minimo de fonte unica
 * Complexidade:  $O((V + E) \log V)$ 
 * Requer pesos nao-negativos.
 *
 * adj[u] = lista de {v, w}: aresta u-v com peso w
 *
 * Retorna vetor dist onde dist[v] = distancia minima de s a v.
 * dist[v] = INF se v e inalcançavel.
 */

67A template<typename T>
7D8 vector<T> dijkstra(const vector<vector<pair<int,T>>>& adj, int s) {
B4C     int n = adj.size();
305     const T INF = numeric_limits<T>::max();
835     vector<T> dist(n, INF);
CFC     priority_queue<pair<T,int>, vector<pair<T,int>>, greater<>> pq;
A93     dist[s] = 0;
78A     pq.push({0, s});
502     while (!pq.empty()) {
5C1         auto [d, u] = pq.top(); pq.pop();
3E1         if (d > dist[u]) continue;
610         for (auto [v, w] : adj[u])
DDE             if (dist[u] + w < dist[v]) {
491                 dist[v] = dist[u] + w;
BF3                 pq.push({dist[v], v});
D74             }
9AE         }
8D7     return dist;
25B }

```

MST (Kruskal)

```

636 #include "../data-structures/dsu.hpp"

/* MST – Arvore Geradora Minima (Kruskal + DSU)
 * Complexidade:  $O(E \log E)$ . Grafo nao-direcionado ponderado.
 *
 * mst(n, edges) → {custo total, arestas usadas}
 * edges: vetor de {peso, u, v}.
 * Se o grafo for desconexo, devolve a floresta geradora minima;
 * cheque used.size() == n-1 para saber se o grafo era conexo.
 *
 * Para arvore geradora MAXIMA: negue os pesos (ou ordene decrescente).
 */

67A template<typename T>
45D pair<T, vector<pair<int,int>>> mst(int n, vector<tuple<T,int,int>> edges) {
FF1     sort(edges.begin(), edges.end());
370     DSU dsu(n);
C80     T total = 0;
24A     vector<pair<int,int>> used;
DC1     for (auto& [w, u, v] : edges)
5DF         if (dsu.join(u, v)) {
532             total += w;
12C             used.push_back({u, v});
88B         }
OCF     return {total, used};
687 }

```

Euler Path

```

/* Caminho/Circuito Euleriano – Hierholzer –  $O(V + E)$ 
 *
 * Condiçoes de existencia:
 * - Grafo nao-direcionado:
 *   - Circuito: todos os vertices com grau par.
 *   - Caminho: exatamente 2 vertices com grau impar (inicio e fim).
 * - Grafo direcionado:
 *   - Circuito: in_degree[v] == out_degree[v] para todo v.
 *   - Caminho: exatamente 1 vertice com out-in=+1 (inicio),
 *               exatamente 1 com in-out=+1 (fim), resto iguais.
 *
 * euler(adj, s) → vetor de vertices do caminho (tamanho E+1).
 * Retorna vetor vazio se nao existe caminho euleriano a partir de s.
 * adj[u] = lista de {v, edge_id}: aresta u-v com indice edge_id.
 * Para grafo nao-direcionado, adicione cada aresta em ambas as direcoes
 * com o mesmo edge_id; o algoritmo marca arestas visitadas pelo id.
 */

E48 vector<int> euler(vector<vector<pair<int,int>>>& adj, int s) {

```

```

229 int E = 0;
A0B for (auto& v : adj) E += v.size();
7E4 E /= 2; // para nao-direcionado; para direcionado, remova o /2

63B vector<bool> used_edge(E, false);
8E1 vector<int> ptr(adj.size(), 0);
DAB vector<int> path, stk = {s};

07D while (!stk.empty()) {
9E9     int v = stk.back();
2C4     bool found = false;
EBE     while (ptr[v] < (int)adj[v].size()) {
FFC         auto [u, eid] = adj[v][ptr[v]++];
A6E         if (!used_edge[eid]) {
A2C             used_edge[eid] = true;
967             stk.push_back(u);
E96             found = true;
C2B             break;
F12         }
BCF     }
8C1     if (!found) {
80A         path.push_back(v);
518         stk.pop_back();
9D9     }
C16 }

867 if ((int)path.size() != E + 1) return {};
3A9 reverse(path.begin(), path.end());
535 return path;
F91 }

```

Lca Sparse Table

```

D09 #include "../data-structures/sparse-table/sparse-table.hpp"

/* LCA (Lowest Common Ancestor) via RMQ
 * Encontra o ancestral comum mais proximo de dois vertices em uma arvore.
 * Complexidade:  $O(N \log N)$  no build,  $O(1)$  por query.
 * Memoria:  $O(N \log N)$ 
 * Requisitos:
 * - Grafo deve ser uma arvore (conexo e aciclico).
 * - Dependencia: data-structures/sparse-table/sparse-table.hpp
 * Metodos:
 * - lca(u, v): Retorna o LCA de u e v.
 * - dist(u, v): Retorna a distancia entre u e v.
 * - is_ancestor(u, v): Verifica se u e ancestral de v.
 * - parent(u): Retorna o pai de u.
 */

542 struct LCANode {
43D     int dep, id;
89D     LCANode(int d = 1e9, int pos = -1) : dep(d), id(pos) {}

5CF     static inline LCANode merge(const LCANode& l, const LCANode& r) {
DC4         return l.dep < r.dep ? l : r;
505     }
DAD };

33E struct LCA {
060     int N;
41B     vector<int> first, tour, d, p;
73B     SparseTable<LCANode> st;

DFC     LCA(const vector<vector<int>>& adj, int root = 0)
A29         : N(adj.size()), first(N), d(N), p(N, -1), st(vector<LCANode>()) {
AA1         tour.reserve(2 * N);
52E         vector<LCANode> nodes;
A7F         nodes.reserve(2 * N);

A12         auto dfs = [&](auto self, int u, int parent_id, int dep) -> void {
73F             p[u] = parent_id;
701             d[u] = dep;
D86             first[u] = tour.size();
FD8             tour.push_back(u);
612             nodes.push_back({dep, (int)tour.size() - 1});
372             for (int v : adj[u]) {
F21                 if (v == parent_id) continue;
207                 self(self, v, u, dep + 1);
FD8             tour.push_back(u);
612             nodes.push_back({dep, (int)tour.size() - 1});
B13         }
252     };
7D9     dfs(dfs, root, -1, 0);
F2E     st = SparseTable<LCANode>(nodes);
186 }

7B9 int parent(int u) {
F60     return p[u];
47F }

310 int lca(int u, int v) {
B63     int l = first[u], r = first[v];
A13     if (l > r) swap(l, r);
369     return tour[st.query(l, r).id];
1E6 }

C11 int dist(int u, int v) {

```

```

05C         return d[u] + d[v] - 2 * d[lca(u, v)];
465     }

F31 bool is_ancestor(int u, int v) {
645     return lca(u, v) == u;
C22 }
FFD };

```

Lca Binary Lifting

```

/* LCA (Lowest Common Ancestor) via Binary Lifting
 * Encontra o ancestral comum mais proximo de dois vertices em uma arvore.
 * Complexidade:  $O(N \log N)$  no build,  $O(\log N)$  por query.
 * Memoria:  $O(N \log N)$ 
 * Requisitos:
 * - Grafo deve ser uma arvore (conexo e aciclico).
 * Metodos:
 * - lca(u, v): Retorna o LCA de u e v.
 * - dist(u, v): Retorna a distancia entre u e v.
 * - is_ancestor(u, v): Verifica se u e ancestral de v.
 * - parent(u): Retorna o pai de u (up[u][0]).
 * - kth_ancestor(u, k): Retorna o k-esimo ancestral de u (-1 se nao existe).
 */

33E struct LCA {
8B7     int N, LOG;
5CE     vector<int> d;
5FE     vector<vector<int>> up;

DFC     LCA(const vector<vector<int>>& adj, int root = 0)
36E         : N(adj.size()), LOG(__lg(N) + 1), d(N), up(N, vector<int>(LOG + 1, -1))...

91B         auto dfs = [&](auto self, int u, int p, int dep) -> void {
701             d[u] = dep;
F1D             up[u][0] = (p == -1) ? u : p;
B16             for (int j = 1; j <= LOG; j++)
4D5                 up[u][j] = up[up[u][j - 1]][j - 1];
372             for (int v : adj[u]) {
730                 if (v == p) continue;
207                 self(self, v, u, dep + 1);
320             }
381         };

7D9     dfs(dfs, root, -1, 0);
0DA }

7B9 int parent(int u) {
826     return up[u][0] == u ? -1 : up[u][0];
579 }

6AE int kth_ancestor(int u, int k) {
472     if (k > d[u]) return -1;
4C8     for (int j = 0; j <= LOG; j++)
518         if ((k >> j) & 1) u = up[u][j];
03F     return u;
51C }

310 int lca(int u, int v) {
4F1     if (d[u] < d[v]) swap(u, v);
38A     int diff = d[u] - d[v];
4C8     for (int j = 0; j <= LOG; j++)
122         if ((diff >> j) & 1) u = up[u][j];
60E     if (u == v) return u;
CAE     for (int j = LOG; j >= 0; j--)
76F         if (up[u][j] != up[v][j]) {
0EF             u = up[u][j];
E4F             v = up[v][j];
D97         }
C15     return up[u][0];
B04 }

C11 int dist(int u, int v) {
05C     return d[u] + d[v] - 2 * d[lca(u, v)];
465 }

F31 bool is_ancestor(int u, int v) {
645     return lca(u, v) == u;
C22 }
F95 };

```

Tarjan

```

/* Tarjan SCC (Strongly Connected Components)
 * Complexidade:  $O(V + E)$ 
 *
 * Campos publicos apos build():
 * comp[v] = id do SCC de v (0-indexado, ordem topologica reversa)
 * scc[i] = lista de vertices do i-esimo SCC
 * dag     = grafo condensado (arestas entre SCCs distintos, sem duplicatas)
 * n_scc   = numero de SCCs
 *
 * Nota: comp[] esta em ordem topologica reversa do DAG condensado,
 * i.e., se ha aresta SCC a → SCC b, entao comp[a] > comp[b].
 * Para ordem topologica direta, percorra de n_scc-1 a 0.
 */

FA3 struct Tarjan {
058     int n, n_scc = 0;

```



```

A41 vector<vector<int>> g, scc, dag;
F7A vector<int> comp;

770 Tarjan(int n) : n(n), g(n), comp(n, -1) {}

224 Tarjan(const vector<vector<int>>& adj)
35F : n(adj.size()), g(adj), comp(adj.size(), -1) { build(); }

58B void add_edge(int u, int v) {
F2B g[u].push_back(v);
C2B }

0A8 void build() {
E7C vector<int> low(n), num(n, -1), stk;
C05 vector<bool> on_stk(n, false);
2DC int timer = 0;

36D function<void(int)> dfs = [&](int v) {
006 low[v] = num[v] = timer++;
800 stk.push_back(v);
48C on_stk[v] = true;
E32 for (int u : g[v]) {
9CE if (num[u] == -1) { dfs(u); low[v] = min(low[v], low[u]); }
52B else if (on_stk[u]) low[v] = min(low[v], num[u]);
38B }
46E if (low[v] == num[v]) {
861 scc.push_back({});
667 while (true) {
D1E int u = stk.back(); stk.pop_back();
635 on_stk[u] = false;
65B comp[u] = n_scc;
588 scc.back().push_back(u);
163 if (u == v) break;
87F }
385 n_scc++;
6CD }
5A5 };

830 for (int i = 0; i < n; i++)
917 if (num[i] == -1) dfs(i);

9E4 dag.assign(n_scc, {});
19E for (int u = 0; u < n; u++)
4FB for (int v : g[u])
C76 if (comp[u] != comp[v])
FC7 dag[comp[u]].push_back(comp[v]);

404 for (auto& adj : dag) {
324 sort(adj.begin(), adj.end());
942 adj.erase(unique(adj.begin(), adj.end()), adj.end());
80E }
BA2 }
6D7 };

```

Articulations

```

/* Pontos de Articulaçao - Tarjan
* Complexidade: O(V + E)
* Grafo nao-direcionado.
*
* Um vertice e ponto de articulacao se sua remocao desconecta o grafo.
*
* Campos publicos apos build():
* is_ap[v] = true se v e ponto de articulacao
* get() = lista de vertices que sao pontos de articulacao
*/

```

```

E5A struct Articulations {
1A8 int n;
789 vector<vector<int>> g;
BC9 vector<bool> is_ap;

620 Articulations(int n) : n(n), g(n), is_ap(n, false) {}

6F5 Articulations(const vector<vector<int>>& adj)
B9D : n(adj.size()), g(adj), is_ap(adj.size(), false) { build(); }

58B void add_edge(int u, int v) {
F2B g[u].push_back(v);
4D6 g[v].push_back(u);
FFF }

0A8 void build() {
AC3 vector<int> num(n, -1), low(n);
2DC int timer = 0;

CF3 function<void(int,int)> dfs = [&](int v, int par) {
006 low[v] = num[v] = timer++;
E07 int children = 0;
E32 for (int u : g[v]) {
403 if (num[u] == -1) {
C5F children++;
412 dfs(u, v);
C80 low[v] = min(low[v], low[u]);
369 if (par == -1 && children > 1) is_ap[v] = true;
183 if (par != -1 && low[u] >= num[v]) is_ap[v] = true;
709 } else if (u != par) {
2D3 low[v] = min(low[v], num[u]);
933 }
}
}

```

```

9F1 }
8CA };

830 for (int i = 0; i < n; i++)
10B if (num[i] == -1) dfs(i, -1);
DEE }

// Retorna lista de vertices que sao pontos de articulacao
D3D vector<int> get() const {
02F vector<int> res;
830 for (int i = 0; i < n; i++)
BA3 if (is_ap[i]) res.push_back(i);
B50 return res;
60A }
A5D };

```

Bridges

```

/* Pontes (Bridges) - Tarjan
* Complexidade: O(V + E)
* Grafo nao-direcionado.
*
* Uma aresta e ponte se sua remocao desconecta o grafo.
*
* Campos publicos apos build():
* bridges = lista de arestas {u, v} que sao pontes
*/

```

```

D44 struct Bridges {
1A8 int n;
789 vector<vector<int>> g;
0CC vector<pair<int,int>> bridges;

B0A Bridges(int n) : n(n), g(n) {}

C65 Bridges(const vector<vector<int>>& adj)
5F7 : n(adj.size()), g(adj) { build(); }

58B void add_edge(int u, int v) {
F2B g[u].push_back(v);
4D6 g[v].push_back(u);
FFF }

0A8 void build() {
AC3 vector<int> num(n, -1), low(n);
2DC int timer = 0;

CF3 function<void(int,int)> dfs = [&](int v, int par) {
006 low[v] = num[v] = timer++;
E32 for (int u : g[v]) {
403 if (num[u] == -1) {
412 dfs(u, v);
C80 low[v] = min(low[v], low[u]);
2FE if (low[u] > num[v]) bridges.push_back({v, u});
FB4 } else if (u != par) {
2D3 low[v] = min(low[v], num[u]);
E66 }
7A3 }
938 };

830 for (int i = 0; i < n; i++)
10B if (num[i] == -1) dfs(i, -1);
A6F }
000 };

```

2-SAT

```

C31 #include "tarjan.hpp"

/* 2-SAT - Complexidade: O(V + E)
*
* Literal x: use i para x_i=true, ~i para x_i=false.
*
* add_impl(x, y) -> x -> y (contrapositiva ~y -> ~x automatica)
* add_or(x, y) -> x v y
* add_xor(x, y) -> exatamente um de x, y e true
* add_iff(x, y) -> x == y
* force(x) -> x = true
* solve() -> {satisfavel, ans[]}; ans[i] = 0 ou 1
*/

```

```

D9D struct TwoSat {
1A8 int n;
0D7 Tarjan t;
DAB vector<int> ans;

46B TwoSat(int n) : n(n), t(2 * n), ans(n, -1) {}

// converte literal para no interno: i >= 0 + 2*i, i < 0 - 2*i-1
5D7 static int node(int x) { return x >= 0 ? 2 * x : -2 * x - 1; }

// implicacao x -> y (e contrapositiva ~y -> ~x)
74A void add_impl(int x, int y) {
D64 int nx = node(x), ny = node(y);
2B2 t.add_edge(nx, ny);
378 t.add_edge(ny ^ 1, nx ^ 1);
255 }

// x v y
F10 void add_or(int x, int y) { add_impl(~x, y); }

```

```

// x XOR y (exatamente um verdadeiro)
C02 void add_xor(int x, int y) { add_or(x, y); add_or(~x, ~y); }

// x == y
721 void add_iff(int x, int y) { add_impl(x, y); add_impl(~x, ~y); }

// forca x = true
5F8 void force(int x) { add_impl(~x, x); }

A8E pair<bool, vector<int>> solve() {
2BD t.build();
E1C ans.assign(n, -1);
603 for (int i = 0; i < n; i++) {
EBA if (t.comp[2 * i] == t.comp[2 * i + 1]) return {false, {}};
049 ans[i] = t.comp[2 * i] < t.comp[2 * i + 1] ? 1 : 0;
EF0 }
997 return {true, ans};
194 }
AC6 };

```

HLD Commutative

```

6A0 #include "../data-structures/segment-tree/segment-tree.hpp"

/* HLD Comutativo (Update Pontual)
* Para operacoes onde A + B = B + A.
* Requisito: data-structures/segment-tree/segment-tree.hpp
*/

```

```

8A0 template<typename NODE>
403 struct HLD {
577 int n, t;
523 vector<int> p, sz, d, head, pos;
115 SegTree<NODE> st;

7EC HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
833 : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {

898 vector<vector<int>> g = adj;
227 d[root] = 0, p[root] = root;

94B auto dfs_sz = [&](auto self, int u) -> void {
267 sz[u] = 1;
839 int best_v = -1, max_sz = -1;
A8C for (auto &v : g[u]) {
567 if (v == p[u]) continue;
ABE d[v] = d[u] + 1; p[v] = u;
033 self(self, v);
CC3 sz[u] += sz[v];
1F6 if (sz[v] > max_sz) {
F66 max_sz = sz[v];
DD0 best_v = v;
D9D }
A7E }
2D5 if (best_v != -1) {
BB6 for (int i = 0; i < (int)g[u].size(); i++) {
F10 if (g[u][i] == best_v && i != 0) {
D76 swap(g[u][0], g[u][i]);
C2B break;
27A }
CAE }
07D }
A11 };

B9B auto dfs_hld = [&](auto self, int u) -> void {
4C8 pos[u] = t++;
BB6 for (int i = 0; i < (int)g[u].size(); i++) {
06E int v = g[u][i];
567 if (v == p[u]) continue;
8D7 head[v] = (i == 0 ? head[u] : v);
033 self(self, v);
D08 }
113 };

A8A dfs_sz(dfs_sz, root);
C07 head[root] = root;
C37 dfs_hld(dfs_hld, root);

759 vector<NODE> base(n);
830 for(int i = 0; i < n; i++)
101 base[pos[i]] = vals[i];
E4E st = SegTree<NODE>(base);
DE8 }

E1B void update(int u, const NODE& val) {
C7A st.update(pos[u], val);
080 }

NODE query_path(int u, int v) {
0F8 NODE res;
ADA while (head[u] != head[v]) {
0E0 if (d[head[u]] > d[head[v]]) swap(u, v);
44B res = NODE::merge(res, st.query(pos[head[v]], pos[v]));
A0D v = p[head[v]];
025 }
41F }
0E0 if (d[u] > d[v]) swap(u, v);
DC7 return NODE::merge(res, st.query(pos[u], pos[v]));
81B }

```

```

51A     NODE query_subtree(int u) {
0CF         return st.query(pos[u], pos[u] + sz[u] - 1);
747     }
058 };

```

HLD Commutative Lazy

```

2F0 #include "../data-structures/segment-tree/lazy-segment-tree.hpp"

```

```

/* HLD Comutativo com Lazy Propagation
 * Para operacoes onde A + B = B + A.
 * Permite updates e queries em caminhos e subarvores.
 * Requisito: data-structures/segment-tree/lazy-segment-tree.hpp
 */

708 template<typename NODE, typename TAG>
403 struct HLD {
577     int n, t;
523     vector<int> p, sz, d, head, pos;
FFF     LazySegmentTree<NODE, TAG> st;

7EC     HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
833     : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {

898     vector<vector<int>> g = adj;
227     d[root] = 0, p[root] = root;

94B     auto dfs_sz = [&](auto self, int u) -> void {
267         sz[u] = 1;
839         int best_v = -1, max_sz = -1;
A8C         for (auto &v : g[u]) {
567             if (v == p[u]) continue;
ABE             d[v] = d[u] + 1; p[v] = u;
033             self(self, v);
CC3             sz[u] += sz[v];
D9D             if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
A7E         }
2D5         if (best_v != -1) {
BB6             for (int i = 0; i < (int)g[u].size(); i++) {
F10                 if (g[u][i] == best_v && i != 0) {
D76                     swap(g[u][0], g[u][i]);
C2B                     break;
27A                 }
CAE             }
07D         }
A11     };

B9B     auto dfs_hld = [&](auto self, int u) -> void {
4C8         pos[u] = t++;
BB6         for (int i = 0; i < (int)g[u].size(); i++) {
06E             int v = g[u][i];
567             if (v == p[u]) continue;
BD7             head[v] = (i == 0 ? head[u] : v);
033             self(self, v);
D08         }
113     };

A8A     dfs_sz(dfs_sz, root);
C07     head[root] = root;
C37     dfs_hld(dfs_hld, root);

759     vector<NODE> base(n);
830     for(int i = 0; i < n; i++)
101         base[pos[i]] = vals[i];
9F8     st = LazySegmentTree<NODE, TAG>(base);
1B8 }

D7C     void update_path(int u, int v, const TAG& tag) {
0E0         while (head[u] != head[v]) {
44B             if (d[head[u]] > d[head[v]]) swap(u, v);
EC8             st.update(pos[head[v]], pos[v], tag);
025             v = p[head[v]];
DD0         }
0E0         if (d[u] > d[v]) swap(u, v);
07A         st.update(pos[u], pos[v], tag);
656     }

0F8     NODE query_path(int u, int v) {
ADA         NODE res;
0E0         while (head[u] != head[v]) {
44B             if (d[head[u]] > d[head[v]]) swap(u, v);
A0D             res = NODE::merge(res, st.query(pos[head[v]], pos[v]));
025             v = p[head[v]];
41F         }
0E0         if (d[u] > d[v]) swap(u, v);
DC7         return NODE::merge(res, st.query(pos[u], pos[v]));
81B     }

F81     void update_subtree(int u, const TAG& tag) {
675         st.update(pos[u], pos[u] + sz[u] - 1, tag);
20D     }

51A     NODE query_subtree(int u) {
0CF         return st.query(pos[u], pos[u] + sz[u] - 1);
747     }
AA5 };

```

HLD Non Commutative

```

6A0 #include "../data-structures/segment-tree/segment-tree.hpp"

```

```

/* HLD Nao-Comutativo (Update Pontual)
 * Para operacoes onde A * B != B * A.
 * Requisito: DoubleNode para gerenciar merges em ambas as direcoes.
 */

8A0 template<typename NODE>
0FF struct DoubleNode {
A30     NODE down, up;
F17     DoubleNode() : down(NODE()), up(NODE()) {}
77F     DoubleNode(const NODE& n) : down(n), up(n) {}
31A     DoubleNode(const NODE& d, const NODE& u) : down(d), up(u) {}

CFE     static inline DoubleNode merge(const DoubleNode& l, const DoubleNode& r) {
2D1         return {
15C             NODE::merge(l.down, r.down),
F17             NODE::merge(r.up, l.up)
5D3         };
809     };
3DC };

8A0 template<typename NODE>
403 struct HLD {
577     int n, t;
256     vector<int> p, sz, d, head, pos, tour;
5B8     SegTree<DoubleNode<NODE>> st;

7EC     HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
7F8     : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), tour(n), st(n) {

898     vector<vector<int>> g = adj;
227     d[root] = 0, p[root] = root;

94B     auto dfs_sz = [&](auto self, int u) -> void {
267         sz[u] = 1;
839         int best_v = -1, max_sz = -1;
A8C         for (auto &v : g[u]) {
567             if (v == p[u]) continue;
ABE             d[v] = d[u] + 1; p[v] = u;
033             self(self, v);
CC3             sz[u] += sz[v];
D9D             if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
A7E         }
2D5         if (best_v != -1) {
BB6             for (int i = 0; i < (int)g[u].size(); i++) {
F10                 if (g[u][i] == best_v && i != 0) {
D76                     swap(g[u][0], g[u][i]);
C2B                     break;
27A                 }
CAE             }
07D         }
A11     };

B9B     auto dfs_hld = [&](auto self, int u) -> void {
4C8         pos[u] = t++;
6EA         tour[pos[u]] = u;
BB6         for (int i = 0; i < (int)g[u].size(); i++) {
06E             int v = g[u][i];
567             if (v == p[u]) continue;
BD7             head[v] = (i == 0 ? head[u] : v);
033             self(self, v);
D08         }
54A     };

A8A     dfs_sz(dfs_sz, root);
C07     head[root] = root;
C37     dfs_hld(dfs_hld, root);

9FC     vector<DoubleNode<NODE>> base(n);
830     for(int i = 0; i < n; i++)
9A8         base[pos[i]] = DoubleNode<NODE>(vals[i]);
19C     st = SegTree<DoubleNode<NODE>>(base);
997 }

E1B     void update(int u, const NODE& val) {
353         st.update(pos[u], DoubleNode<NODE>(val));
443     }

NODE query_path(int u, int v) {
NODE L, R;
while (head[u] != head[v]) {
if (d[head[u]] > d[head[v]]) {
L = NODE::merge(L, st.query(pos[head[u]], pos[u]).up);
u = p[head[u]];
} else {
R = NODE::merge(st.query(pos[head[v]], pos[v]).down, R);
v = p[head[v]];
}
}
if (d[u] > d[v]) L = NODE::merge(L, st.query(pos[v], pos[u]).up);
else R = NODE::merge(st.query(pos[u], pos[v]).down, R);
return NODE::merge(L, R);
}

NODE query_subtree(int u) {

```

```

DE6         return st.query(pos[u], pos[u] + sz[u] - 1).down;
588     }
98B };

```

HLD Non Commutative Lazy

```

2F0 #include "../data-structures/segment-tree/lazy-segment-tree.hpp"

```

```

/* HLD Nao-Comutativo com Lazy Propagation
 * Para operacoes onde A + B != B + A (ex: Matrices, Funcoes Lineares).
 * Requisito: DoubleNode que guarda versoes 'normal' e 'invertida' do merge.
 */

8A0 template<typename NODE>
0FF struct DoubleNode {
A30     NODE down, up;
F17     DoubleNode() : down(NODE()), up(NODE()) {}
77F     DoubleNode(const NODE& n) : down(n), up(n) {}
31A     DoubleNode(const NODE& d, const NODE& u) : down(d), up(u) {}

CFE     static inline DoubleNode merge(const DoubleNode& l, const DoubleNode& r) {
2D1         return {
15C             NODE::merge(l.down, r.down),
F17             NODE::merge(r.up, l.up)
5D3         };
809     };

BBF     template<typename TAG>
3CC     void apply(const TAG& t, int l, int r) {
B08         down.apply(t, l, r);
2BF         up.apply(t, l, r);
8FB     }
87B };

708 template<typename NODE, typename TAG>
403 struct HLD {
577     int n, t;
523     vector<int> p, sz, d, head, pos;
C94     LazySegmentTree<DoubleNode<NODE>, TAG> st;

7EC     HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
833     : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {

898     vector<vector<int>> g = adj;
227     d[root] = 0, p[root] = root;

94B     auto dfs_sz = [&](auto self, int u) -> void {
267         sz[u] = 1;
839         int best_v = -1, max_sz = -1;
A8C         for (auto &v : g[u]) {
567             if (v == p[u]) continue;
ABE             d[v] = d[u] + 1; p[v] = u;
033             self(self, v);
CC3             sz[u] += sz[v];
D9D             if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
A7E         }
2D5         if (best_v != -1) {
BB6             for (int i = 0; i < (int)g[u].size(); i++) {
F10                 if (g[u][i] == best_v && i != 0) {
D76                     swap(g[u][0], g[u][i]);
C2B                     break;
27A                 }
CAE             }
07D         }
A11     };

B9B     auto dfs_hld = [&](auto self, int u) -> void {
4C8         pos[u] = t++;
BB6         for (int i = 0; i < (int)g[u].size(); i++) {
06E             int v = g[u][i];
567             if (v == p[u]) continue;
BD7             head[v] = (i == 0 ? head[u] : v);
033             self(self, v);
D08         }
113     };

A8A     dfs_sz(dfs_sz, root);
C07     head[root] = root;
C37     dfs_hld(dfs_hld, root);

9FC     vector<DoubleNode<NODE>> base(n);
B89     for(int i = 0; i < n; i++) base[pos[i]] = DoubleNode<NODE>(vals[i]);
54E     st = LazySegmentTree<DoubleNode<NODE>, TAG>(base);
B60 }

D7C     void update_path(int u, int v, const TAG& tag) {
0E0         while (head[u] != head[v]) {
BF1             if (d[head[u]] > d[head[v]]) {
D76                 st.update(pos[head[u]], pos[u], tag);
139                 u = p[head[u]];
D79             } else {
EC8                 st.update(pos[head[v]], pos[v], tag);
025                 v = p[head[v]];
E8B             }
15B         }
B83         if (d[u] > d[v]) st.update(pos[v], pos[u], tag);
052         else st.update(pos[u], pos[v], tag);
855     }
}

```

```

0F8     NODE query_path(int u, int v) {
34B         NODE L, R;
0E0         while (head[u] != head[v]) {
BF1             if (d[head[u]] > d[head[v]]) {
0F3                 L = NODE::merge(L, st.query(pos[head[u]], pos[u]).up);
139                 u = p[head[u]];
3B2             } else {
B5C                 R = NODE::merge(st.query(pos[head[v]], pos[v]).down, R);
025                 v = p[head[v]];
7D9             }
D8B         }
152         if (d[u] > d[v]) {
7E5             L = NODE::merge(L, st.query(pos[v], pos[u]).up);
D6C         } else {
806             R = NODE::merge(st.query(pos[u], pos[v]).down, R);
8A1         }
85E         return NODE::merge(L, R);
EB1     }

F81     void update_subtree(int u, const TAG& tag) {
675         st.update(pos[u], pos[u] + sz[u] - 1, tag);
20D     }

51A     NODE query_subtree(int u) {
DE6         return st.query(pos[u], pos[u] + sz[u] - 1).down;
588     }
068 };

```

Dinic

```

/* Dinic (Max Flow)
* Complexidade: O(V^2 * E) geral, O(E * sqrt(V)) em grafos unitarios.
* Memoria: O(V + E)
*
* Metodos:
*   add_edge(u, v, cap)      → aresta u-v com capacidade cap
*   add_edge(u, v, cap, rcap) → idem com aresta reversa rcap (nao-direcionado)
*   flow(s, t)               → fluxo maximo de s a t
*   min_cut(s)               → apos flow(), vetor onde [i]=true se i esta no lado da fonte
*/

67A template<typename T>
14D struct Dinic {
E9B     struct Edge {
C63         int to, rev;
4AE         T cap;
001     };

060     int N;
ED3     vector<vector<Edge>> g;
538     vector<int> level, iter;

241     Dinic(int n) : N(n), g(n), level(n), iter(n) {}

D8F     void add_edge(int u, int v, T cap, T rcap = 0) {
25C         g[u].push_back({v, (int)g[v].size(), cap});
ACA         g[v].push_back({u, (int)g[u].size() - 1, rcap});
05B     }

123     bool bfs(int s, int t) {
224         fill(level.begin(), level.end(), -1);
26A         queue<int> q;
EC8         level[s] = 0;
08B         q.push(s);
14D         while (!q.empty()) {
0E6             int v = q.front(); q.pop();
24E             for (auto& e : g[v])
466                 if (e.cap > 0 && level[e.to] < 0) {
BD7                     level[e.to] = level[v] + 1;
6F4                     q.push(e.to);
A33                 }
FF9             }
FCB         return level[t] >= 0;
56A     }

D20     T dfs(int v, int t, T f) {
704         if (v == t) return f;
6E7         for (int& i = iter[v]; i < (int)g[v].size(); i++) {
63C             Edge& e = g[v][i];
2C2             if (e.cap > 0 && level[v] < level[e.to]) {
054                 T d = dfs(e.to, t, min(f, e.cap));
1A3                 if (d > 0) {
8FC                     e.cap -= d;
772                     g[e.to][e.rev].cap += d;
BE2                     return d;
18D                 }
57D             }
E13         }
BB3         return 0;
182     }

903     T flow(int s, int t) {
19A         T res = 0;
8CE         while (bfs(s, t)) {
736             fill(iter.begin(), iter.end(), 0);
A96             T d;

```

```

AE9         while ((d = dfs(s, t, numeric_limits<T>::max())) > 0)
337             res += d;
91B         }
B50         return res;
CE8     }

8B0     vector<bool> min_cut(int s) {
F71         vector<bool> vis(N, false);
26A         queue<int> q;
451         vis[s] = true;
08B         q.push(s);
14D         while (!q.empty()) {
0E6             int v = q.front(); q.pop();
24E             for (auto& e : g[v])
F04                 if (e.cap > 0 && !vis[e.to]) {
1E8                     vis[e.to] = true;
6F4                     q.push(e.to);
A56                 }
607             }
C81             return vis;
71F         }
993 };

```

Min Cost Max Flow

```

/* Min-Cost Max-Flow (SPFA/Bellman-Ford based)
* Complexidade: O(V * E * flow) no pior caso; na pratica muito mais rapido.
* Memoria: O(V + E)
*
* Metodos:
*   add_edge(u, v, cap, cost)      → aresta u-v com cap e custo
*   add_edge(u, v, cap, rcap, cost) → idem com aresta reversa rcap
*   flow(s, t)                    → {fluxo maximo, custo minimo} de s a t
*   flow(s, t, max_flow)          → idem limitando o fluxo total a max_flow
*
* Observacoes:
*   - Suporta custos negativos (usa SPFA em vez de Dijkstra).
*   - Sem custos negativos, prefira Dijkstra+potenciais: O(E log V)/aug.
*   - 0 custo retornado e o custo minimo para enviar o fluxo maximo.
*/

39C template<typename Cap, typename Cost>
6F3 struct MCMF {
E9B     struct Edge {
C63         int to, rev;
2E2         Cap cap;
CB9         Cost cost;
BFF     };

060     int N;
ED3     vector<vector<Edge>> g;

7E6     MCMF(int n) : N(n), g(n) {}

251     void add_edge(int u, int v, Cap cap, Cost cost, Cap rcap = 0) {
C5D         g[u].push_back({v, (int)g[v].size(), cap, cost});
D90         g[v].push_back({u, (int)g[u].size() - 1, rcap, -cost});
200     }

pair<Cap, Cost> flow(int s, int t,
                    Cap max_flow = numeric_limits<Cap>::max()) {
    Cap total_flow = 0;
    8AC     Cost total_cost = 0;
    8F8

    8F6     while (max_flow > 0) {
715         vector<Cost> dist(N, numeric_limits<Cost>::max());
7A2         vector<bool> in_queue(N, false);
FF9         vector<int> prev_node(N, -1), prev_edge(N, -1);

A93         dist[s] = 0;
26A         queue<int> q;
08B         q.push(s);
6FD         in_queue[s] = true;

14D         while (!q.empty()) {
0E6             int v = q.front(); q.pop();
F19             in_queue[v] = false;
775             for (int i = 0; i < (int)g[v].size(); i++) {
027                 auto& e = g[v][i];
AB4                 if (e.cap > 0 && dist[v] + e.cost < dist[e.to]) {
9B1                     dist[e.to] = dist[v] + e.cost;
3C0                     prev_node[e.to] = v;
2E2                     prev_edge[e.to] = i;
5CA                     if (!in_queue[e.to]) {
304                         in_queue[e.to] = true;
6F4                         q.push(e.to);
8A6                     }
5E7                 }
168             }
A8B         }

472         if (dist[t] == numeric_limits<Cost>::max()) break;

1E4         Cap pushed = max_flow;
8CE         for (int v = t; v != s; v = prev_node[v])
31C             pushed = min(pushed, g[prev_node[v]][prev_edge[v]].cap);
879         for (int v = t; v != s; v = prev_node[v]) {

```

```

49B         auto& e = g[prev_node[v]][prev_edge[v]];
A04         e.cap -= pushed;
B9A         g[v][e.rev].cap += pushed;
F38     }

3BD         total_flow += pushed;
6CE         total_cost += pushed * dist[t];
12C         max_flow -= pushed;
CC5     }

570     return {total_flow, total_cost};
D1A }
0B4 };

```

Bipartite Matching

```

0AA #include "dinic.hpp"

/* Bipartite Matching via Dinic
* Matching maximo em grafo bipartido.
* Complexidade: O(E * sqrt(V))
*
* Vertices esquerda: [0, n), direita: [n, n+m).
*
* Campos publicos apos match():
*   size = tamanho do matching maximo
*   match_l[i] = vertice da direita matched com i (-1 se nao matched)
*   match_r[j] = vertice da esquerda matched com j (-1 se nao matched)
*
* Metodos:
*   add_edge(i, j) → aresta entre esquerda i e direita j
*   match()       → executa o matching, retorna tamanho
*/

878 struct BipartiteMatching {
14E     int n, m;
690     Dinic<int> g;
3AE     int S, T;
2AF     vector<int> match_l, match_r;
689     int size = 0;

1FD     BipartiteMatching(int n, int m)
104         : n(n), m(m), g(n + m + 2),
75F             S(n + m), T(n + m + 1),
779             match_l(n, -1), match_r(m, -1) {
1D3         for (int i = 0; i < n; i++) g.add_edge(S, i, 1);
CC1         for (int j = 0; j < m; j++) g.add_edge(n + j, T, 1);
B60     }

C5E     void add_edge(int i, int j) {
3EC         g.add_edge(i, n + j, 1);
EE4     }

274     int match() {
5D7         size = g.flow(S, T);
830         for (int i = 0; i < n; i++)
883             for (auto& e : g.g[i])
772                 if (e.to != S && e.cap == 0) {
846                     match_l[i] = e.to - n;
78E                     match_r[e.to - n] = i;
FE7                 }
1C6                 return size;
B47             }
CFA };

```

Math

Mint

```

/* mint - Inteiro modular
* Complexidade: O(1) por operacao, O(log mod) para inv()
*
* template<typename T, T mod>
*
* Uso comum:
*   using mint = Mint<int, 998244353>;
*   using mll = Mint<ll, 998244353LL>;
*
* Operacoes: +, -, *, /, ==, !=, <, <=
*   inv() → inverso modular (mod deve ser primo)
*/

2DB template<typename T, T mod>
4D0 struct Mint {
CA0     T v;
926     Mint(T v = 0) : v(((v % mod) + mod) % mod) {}

E93     Mint operator+(Mint o) const { return v+o.v >= mod ? v+o.v-mod : v+o.v; }
66C     Mint operator-(Mint o) const { return v-o.v < 0 ? v-o.v+mod : v-o.v; }
3B1     Mint operator*(Mint o) const { return (ll)v * o.v % mod; }
5B6     Mint operator/(Mint o) const { return *this * o.inv(); }

96C     Mint& operator+=(Mint o) { return *this = *this + o; }
BB0     Mint& operator-=(Mint o) { return *this = *this - o; }
926     Mint& operator*=(Mint o) { return *this = *this * o; }

```



```

E1C    Mint& operator!=(Mint o) { return *this == *this / o; }
8D1    bool operator==(Mint o) const { return v == o.v; }
587    bool operator!=(Mint o) const { return v != o.v; }
A45    explicit operator T() const { return v; }

EC5    Mint inv() const {
F05        T a = v, b = mod, x = 1, y = 0;
367        while (b) { T q = a/b; a -= q*b; swap(a,b); x -= q*y; swap(x,y); }
EA5        return x;
0CF    }

09A    friend ostream& operator<<(ostream& os, Mint m) { return os << m.v; }
AAA    };

```

Fast Exponentiation

```

/* fexp - Exponenciacao rapida modular
* Complexidade: O(log b)
*
* fexp(a, b, mod) -> a^b % mod
* modinv(a, mod) -> a^{-1} % mod (mod deve ser primo; usa Fermat)
*
* Usa __int128 no produto: seguro para mod ate ~9.2·10^{18}.
*/

42B ll fexp(ll a, ll b, ll mod) {
7E3    a %= mod; if (a < 0) a += mod;
CE0    ll res = 1;
C71    for (; b > 0; b >>= 1, a = (__int128)a * a % mod)
D47        if (b & 1) res = (__int128)res * a % mod;
B50    return res;
A29 }

64B ll modinv(ll a, ll mod) { return fexp(a, mod - 2, mod); }

```

Sieve

```

/* Crivo de Eratostenes
* Complexidade: O(N log log N)
*
* sieve(n) -> vetor com todos os primos ate n (inclusive).
*/

705 vector<int> sieve(int n) {
EFA    vector<bool> is_prime(n + 1, true);
FD3    vector<int> primes;
19E    is_prime[0] = is_prime[1] = false;
8A9    for (int p = 2; p <= n; p++) {
29C        if (is_prime[p]) {
BCE            primes.push_back(p);
908            if ((long long)p * p <= n)
4C7                for (int i = p * p; i <= n; i += p)
F10                    is_prime[i] = false;
FFE        }
5D1    }
A20    return primes;
81C }

/* Crivo de menor fator primo (SPF) - O(N log log N) build
* spf[i] = menor primo que divide i (spf[i] == i se i e primo, spf[0]=spf[1]=...
* Permite fatorar qualquer x <= n em O(log x).
*
* auto spf = spf_sieve(n);
* factor(x, spf) -> vetor {primo, expoente} em ordem crescente de primo.
*/

830 vector<int> spf_sieve(int n) {
78F    vector<int> spf(n + 1);
2B8    iota(spf.begin(), spf.end(), 0);
F9A    for (int i = 2; (long long)i * i <= n; i++)
DD0        if (spf[i] == i)
C0A            for (int j = i * i; j <= n; j += i)
480                if (spf[j] == j) spf[j] = i;
D4F    return spf;
398 }

4FF vector<pair<int,int>> factor(int x, const vector<int>& spf) {
490    vector<pair<int,int>> f;
40E    while (x > 1) {
239        int p = spf[x], e = 0;
E87        while (x % p == 0) { x /= p; e++; }
6BF        f.push_back({p, e});
760    }
ABE    return f;
B2C }

/* Todos os divisores de x (fora de ordem) - O(d(x)) por consulta.
* x precisa estar dentro do crivo (x <= n, o mesmo n de spf_sieve).
* Gera a partir da fatoracao SPF. Faca sort() no retorno se precisar ordenado.
*/

17D vector<int> divisors(int x, const vector<int>& spf) {
FC1    vector<int> divs = {1};
D07    for (auto [p, e] : factor(x, spf)) {
F49        int sz = divs.size(), pk = 1;
6E5        for (int k = 0; k < e; k++) {

```

```

DC5        pk *= p;
BA1        for (int i = 0; i < sz; i++) divs.push_back(divs[i] * pk);
FB5        }
261    }
81C    return divs;
C23 }

```

Euler Phi

```

/* Crivo Linear - O(N)
*
* linear_sieve(n) -> (primes, phi)
* primes : todos os primos ate n (inclusive)
* phi    : phi[i] = φ(i) para i em [0, n]
*/

9A7 pair<vector<int>, vector<int>> linear_sieve(int n) {
303    vector<int> phi(n + 1, 0), primes;
614    vector<bool> composite(n + 1, false);
678    phi[1] = 1;
508    for (int i = 2; i <= n; i++) {
C54        if (!composite[i]) {
E74            primes.push_back(i);
ABD            phi[i] = i - 1;
6DB        }
3B9        for (int p : primes) {
0E8            if ((long long)i * p > n) break;
847            composite[i * p] = true;
569            if (i % p == 0) {
8EC                phi[i * p] = phi[i] * p;
C2B                break;
792            } else {
812                phi[i * p] = phi[i] * (p - 1);
988            }
A0F        }
8CB    }
6EF    return {primes, phi};
158 }

```

Number Theory

```

/* GCD / LCM - O(log min(a, b)) */

225 ll gcd(ll a, ll b) { return b ? gcd(b, a % b) : a; }
15C ll lcm(ll a, ll b) { return a / gcd(a, b) * b; }

/* Divisores de n - O(sqrt(N))
* Retorna lista de divisores em ordem crescente.
*/

5FA vector<ll> divisors(ll n) {
17E    vector<ll> lo, hi;
148    for (ll i = 1; i * i <= n; i++) {
C06        if (n % i == 0) {
1EB            lo.push_back(i);
9C3            if (i != n / i) hi.push_back(n / i);
B5B        }
2DE    }
FC1    reverse(hi.begin(), hi.end());
3AF    lo.insert(lo.end(), hi.begin(), hi.end());
253    return lo;
B9C }

/* Fatoracao em primos - O(sqrt(N))
* Retorna lista de {primo, expoente} em ordem crescente.
*/

7E3 vector<pair<ll,int>> factorize(ll n) {
618    vector<pair<ll,int>> factors;
D2E    for (ll p = 2; p * p <= n; p++) {
80A        if (n % p == 0) {
A01            int exp = 0;
6C9            while (n % p == 0) { n /= p; exp++; }
13A            factors.push_back({p, exp});
2E9        }
47D    }
992    if (n > 1) factors.push_back({n, 1});
FA9    return factors;
92A }

```

```

/* CRT - Teorema Chines dos Restos - O(log min(m1, m2))
* Retorna {x, lcm(m1,m2)} tal que x = a1 (mod m1) e x = a2 (mod m2).
* Retorna {0, 0} se nao ha solucao (m1, m2 nao coprimos e incompativeis).
*/

8A9 pair<ll,ll> crt(ll a1, ll m1, ll a2, ll m2) {
8F7    ll g = gcd(m1, m2);
916    if ((a2 - a1) % g != 0) return {0, 0};
9C3    ll l = m1 / g * m2;
D0B    ll a = m1/g, b = m2/g, x = 1, y = 0;
E58    for (ll ta = a, tb = b; tb; ) {
146        ll q = ta/tb; ta -= q*tb; swap(ta,tb); x -= q*y; swap(x,y);
54E    }
581    x = (x % (m2/g) + m2/g) % (m2/g);
EE5    ll r = (a1 + m1 * ((a2 - a1) / g % (m2/g) * x % (m2/g))) % l;
D27    return {(r + l) % l, l};

```

```

F7F }

/* Fatoracao usando lista de primos pre-computada - O(n(sqrt(N)))
* Use quando ja tiver rodado sieve(sqrt(n)) antes de chamar para varios n.
* primos deve conter todos os primos ate sqrt(n).
*/

E6D vector<pair<ll,int>> factorize(ll n, const vector<int>& primes) {
618    vector<pair<ll,int>> factors;
3B9    for (int p : primes) {
851        if ((ll)p * p > n) break;
80A        if (n % p == 0) {
A01            int exp = 0;
6C9            while (n % p == 0) { n /= p; exp++; }
13A            factors.push_back({p, exp});
2E9        }
17C    }
992    if (n > 1) factors.push_back({n, 1});
FA9    return factors;
490 }

```

Miller Rabin

```

/* Miller-Rabin + Pollard Rho
*
* is_prime(n) -> O(log^2 n), deterministico para n < 3.3·10^{18}
* factorize(n) -> O(n^{1/4} log n) esperado; fatores com repeticao
*
* Use sort + unique para fatores distintos, ou agrupe por expoente.
*/

7D4 ll mulmod(ll a, ll b, ll m) { return (__int128)a * b % m; }

0A0 bool is_prime(ll n) {
5A7    if (n < 2) return false;
0A8    if (n == 2 || n == 3 || n == 5 || n == 7) return true;
3CE    if (n % 2 == 0 || n % 3 == 0) return false;
B87    ll d = n - 1; int r = 0;
5B7    while (d % 2 == 0) d /= 2, r++;
C12    for (ll a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
89B        if (a >= n) continue;
8C8        ll x = 1, base = a % n, exp = d;
006        for (; exp > 0; exp >>= 1, base = mulmod(base, base, n))
5E6            if (exp & 1) x = mulmod(x, base, n);
8A3        if (x == 1 || x == n - 1) continue;
30D        bool composite = true;
21B        for (int i = 0; i < r - 1; i++) {
F38            x = mulmod(x, x, n);
789            if (x == n - 1) { composite = false; break; }
4C7        }
4F2        if (composite) return false;
C63    }
8A6    return true;
462 }

711 ll pollard_rho(ll n) {
55A    if (n % 2 == 0) return 2;
2BC    ll x = rand() % (n - 2) + 2, y = x, c = rand() % (n - 1) + 1, d = 1;
4EB    while (d == 1) {
C17        x = (mulmod(x, x, n) + c) % n;
4EE        y = (mulmod(y, y, n) + c) % n;
4EE        y = (mulmod(y, y, n) + c) % n;
DD8        d = __gcd(abs(x - y), n);
565    }
9EC    return d == n ? pollard_rho(n) : d;
5D7 }

6C2 vector<ll> factorize(ll n) {
1B9    if (n == 1) return {};
970    if (is_prime(n)) return {n};
B32    ll d = pollard_rho(n);
F9F    auto a = factorize(d), b = factorize(n / d);
19E    a.insert(a.end(), b.begin(), b.end());
3F5    return a;
E13 }

```

Matrix

```

/* Matrix - Matriz quadrada com multiplicacao e exponenciacao rapida
* Complexidade: O(n^3 log k) para M^k
*
* Use com mint/mll para operacoes modulares automaticas.
*
* Matrix<T> M(n) -> identidade nxn
* Matrix<T> M(n,0) -> matriz nxn zerada
* M[i][j] -> acesso direto
* A * B -> multiplicacao O(n^3)
* M ^ k -> M^k por exponenciacao rapida
*/

67A template<typename T>
BF3 struct Matrix {
1A8    int n;
C24    vector<vector<T>> mat;
C62    Matrix(int n, int identity = 1) : n(n), mat(n, vector<T>(n, T(0))) {

```

```

603         if (identity) for (int i = 0; i < n; i++) mat[i][i] = T(1);
68A     }
90D     vector<T>& operator[](int i) { return mat[i]; }
699     const vector<T>& operator[](int i) const { return mat[i]; }
AFB     Matrix operator*(const Matrix& rhs) const {
83B         Matrix res(n, 0);
830         for (int i = 0; i < n; i++)
23D             for (int k = 0; k < n; k++) if (mat[i][k] != T(0))
F90                 for (int j = 0; j < n; j++)
C55                     res[i][j] = res[i][j] + mat[i][k] * rhs.mat[k][j];
B50         return res;
893     }
C99     Matrix operator^(long long b) const {
2DF         Matrix res(n), a = *this;
0EB         for (; b > 0; b >= 1, a = a * a)
FC2             if (b & 1) res = res * a;
B50         return res;
154     }
69C };

```

Gaussian Elimination

```

/* Eliminação Gaussiana (RREF) – O(N^2 * M)
* Funciona com double ou Mint<T, mod>.
*
* gauss(A, b) → resolve Ax = b.
* Retorna {x, true} se única, {x, false} se inconsistente/sub-determinado.
*/

67A template<typename T>
5D7 pair<vector<T>, bool> gauss(vector<vector<T>> A, vector<T> b) {
FE3     int n = A.size();
3E0     for (int i = 0; i < n; i++) A[i].push_back(b[i]);
3D7     int m = n + 1, rank = 0;
079     for (int col = 0; col < n && rank < n; col++) {
9D3         int sel = rank;
F97         for (int i = rank + 1; i < n; i++)
E75             if (abs(A[i][col]) > abs(A[sel][col])) sel = i;
9E6         if (!(bool)A[sel][col]) continue;
D8D         swap(A[sel], A[rank]);
B46         T iv = T(1) / A[rank][col];
C24         for (int j = col; j < m; j++) A[rank][j] = A[rank][j] * iv;
603         for (int i = 0; i < n; i++) {
BC7             if (i == rank || !(bool)A[i][col]) continue;
BEF             T f = A[i][col];
BAC             for (int j = col; j < m; j++) A[i][j] = A[i][j] - f * A[rank][j];
371         }
56E         rank++;
3F2     }
175     if (rank < n) return {{}, false};
A1B     vector<T> x(n);
B95     for (int i = 0; i < n; i++) x[i] = A[i].back();
079     return {x, true};
724 }

```

Linear Basis

```

/* Linear Basis (Base Linear XOR)
* Complexidade: O(B) por operação, onde B = número de bits (63 para ll)
* Memória: O(B)
*
* Metodos:
* insert(x) → insere x; retorna true se x é linearmente independente
* contains(x) → true se x é XOR de algum subconjunto da base
* max_xor(x=0) → maior XOR de x com qualquer subconjunto da base
* min_xor() → menor XOR não-nulo possível (0 se base incompleta)
* size() → número de elementos linearmente independentes
* merge(other) → une duas bases (insere todos os elementos de other)
*
* Observacoes:
* - Base em forma reduzida (cada bit pivot aparece em apenas uma linha).
* - Para k-esimo XOR crescente, use reduce() e trate como número binário.
*/

538 template<typename T = ll, int B = 63>
F45 struct LinearBasis {
E9C     array<T, B + 1> basis{};
1FC     int sz = 0;

220     bool insert(T x) {
C8C         for (int i = B; i >= 0; i--) {
760             if (!(x >> i) & 1) continue;
617             if (!basis[i]) {
C6C                 basis[i] = x;
EB9                 sz++;
8A6                 return true;
50C             }
6B0             x ^= basis[i];
8A2         }
D1F         return false;
CD7     }

```

```

394     bool contains(T x) const {
C8C         for (int i = B; i >= 0; i--) {
760             if (!(x >> i) & 1) continue;
260             if (!basis[i]) return false;
6B0             x ^= basis[i];
1A9         }
8A6         return true;
189     }

C3B     T max_xor(T x = 0) const {
086         for (int i = B; i >= 0; i--)
EC9             x = max(x, x ^ basis[i]);
EA5         return x;
74B     }

BBB     T min_xor() const {
F56         for (int i = 0; i <= B; i++)
15D             if (basis[i]) return basis[i];
BB3         return 0;
5FC     }

21A     int size() const { return sz; }

AC0     void merge(const LinearBasis& other) {
086         for (int i = B; i >= 0; i--)
D67             if (other.basis[i]) insert(other.basis[i]);
1CA     }

/* Reduz a base para forma escalonada reduzida (cada pivot tem 1 bit).
* Necessário para consultas de k-esimo XOR.
* Após reduce(), qualquer subconjunto gera um XOR distinto:
* e possível enumerar todos os 2^sz XORs possíveis.
*/

93A     void reduce() {
C8C         for (int i = B; i >= 0; i--) {
F14             if (!basis[i]) continue;
E36             for (int j = i + 1; j <= B; j++)
7E7                 if ((basis[j] >> i) & 1) basis[j] ^= basis[i];
CA5         }
563     }

/* k-esimo menor XOR (0-indexado) entre todos os 2^sz subconjuntos.
* Requer reduce() antes. Se sz < B+1, XOR=0 = subconjunto vazio.
* k=0 → 0; k=1 → menor XOR não-nulo, etc.
*/

0B4     T kth(ll k) const {
E55         vector<T> elems;
F56         for (int i = 0; i <= B; i++)
210             if (basis[i]) elems.push_back(basis[i]);
// elems está em ordem crescente de bit mais significativo
if (k >= (1LL << (int)elems.size())) return -1; // k fora do range
T res = 0;
for (int i = 0; i < (int)elems.size(); i++)
if ((k >> i) & 1) res ^= elems[i];
return res;
}

D55     }
19A     DE8 };

```

Unimodal Search

```

/* Busca unimodal – f deve ser unimodal no intervalo dado.
*
* ternary_min/max(lo, hi, f) → inteiros, O(log N)
* golden_min/max(lo, hi, f, iters=200) → contínuo, O(iters); 200 iters = 1e-30
*/

// Ternary search para mínimo em inteiros [lo, hi]
398 template<typename F>
209 long long ternary_min(long long lo, long long hi, F f) {
960     while (hi - lo > 2) {
C9D         long long m1 = lo + (hi - lo) / 3;
A33         long long m2 = hi - (hi - lo) / 3;
2EE         if (f(m1) <= f(m2)) hi = m2;
F67         else lo = m1;
885     }
52F     long long best = lo;
35B     for (long long x = lo + 1; x <= hi; x++)
EB8         if (f(x) < f(best)) best = x;
F26     return best;
DB5 }

// Ternary search para máximo em inteiros [lo, hi]
398 template<typename F>
E4E long long ternary_max(long long lo, long long hi, F f) {
960     while (hi - lo > 2) {
C9D         long long m1 = lo + (hi - lo) / 3;
A33         long long m2 = hi - (hi - lo) / 3;
750         if (f(m1) >= f(m2)) hi = m2;
F67         else lo = m1;
344     }
52F     long long best = lo;
35B     for (long long x = lo + 1; x <= hi; x++)
592         if (f(x) > f(best)) best = x;
F26     return best;
2AD }

```

```

// Golden section search para mínimo em contínuo [lo, hi]
398 template<typename F>
938 double golden_min(double lo, double hi, F f, int iters = 200) {
84E     const double phi = (sqrt(5.0) - 1.0) / 2.0; // = 0.618
0DD     double m1 = hi - phi * (hi - lo);
86C     double m2 = lo + phi * (hi - lo);
87F     double f1 = f(m1), f2 = f(m2);
5B6     for (int i = 0; i < iters; i++) {
F4D         if (f1 < f2) {
D90             hi = m2; m2 = m1; f2 = f1;
251             m1 = hi - phi * (hi - lo); f1 = f(m1);
735         } else {
86E             lo = m1; m1 = m2; f1 = f2;
9FA             m2 = lo + phi * (hi - lo); f2 = f(m2);
E4A         }
DC3     }
E67     return (lo + hi) / 2.0;
2D2 }

// Golden section search para máximo em contínuo [lo, hi]
398 template<typename F>
CEA double golden_max(double lo, double hi, F f, int iters = 200) {
8ED     return golden_min(lo, hi, [&](double x) { return -f(x); }, iters);
2E7 }

```

FFT

```

316 using CD = complex<ld>;

/* FFT – Fast Fourier Transform
* Complexidade: O(N log N)
*
* fft(a) → transforma a[] no lugar (tamanho deve ser potência de 2)
* conv(a, b) → convolução: c[k] = Σ a[i]*b[k-i]
*
* Precisão: seguro se (Σa²+Σb²)*log₂N < 9·10¹⁴.
* Para coeficientes grandes ou mod, use fft-mod.hpp ou ntt.hpp.
*/

B4C void fft(vector<CD>& a) {
A5B     int n = a.size(), L = 31 - __builtin_clz(n);
F82     static vector<complex<long double>> R(2, 1);
6B4     static vector<CD> rt(2, 1);
AD8     for (static int k = 2; k < n; k *= 2) {
411         auto x = polar(1.0L, acos(-1.0L) / k);
E92         R.resize(n); rt.resize(n);
1D3         for (int i = k; i < 2*k; i++)
CD4             rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
040     }

808     vector<int> rev(n);
5EB     for (int i = 0; i < n; i++) rev[i] = (rev[i/2] | (i&1) << L) / 2;
EE4     for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);

657     for (int k = 1; k < n; k *= 2)
1E5         for (int i = 0; i < n; i += 2*k)
0C2             for (int j = 0; j < k; j++) {
CD2                 auto x = (ld*)&rt[j+k], y = (ld*)&a[i+j+k];
219                 CD z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
20A                 a[i+j+k] = a[i+j] - z;
1B0                 a[i+j] += z;
707             }
F60 }

17B vector<ld> conv(const vector<ld>& a, const vector<ld>& b) {
F88     if (a.empty() || b.empty()) return {};
BBB     vector<ld> res(a.size() + b.size() - 1);
E9A     int n = 1 << (32 - __builtin_clz(res.size()));
576     vector<CD> in(n), out(n);
F83     copy(begin(a), end(a), begin(in));
D38     for (int i = 0; i < (int)b.size(); i++) in[i].imag(b[i]);
21A     fft(in);
11C     for (auto& x : in) x *= x;
2FC     for (int i = 0; i < n; i++) out[i] = in[-i&(n-1)] - conj(in[i]);
3D7     fft(out);
74E     for (int i = 0; i < (int)res.size(); i++) res[i] = imag(out[i]) / (4*n);
B50     return res;
B6D }

```

FFT Mod

```

240 #include "fft.hpp"

/* FFT Mod – Convolução modular com mod arbitrário
* Complexidade: O(N log N) (~2x mais lento que FFT ou NTT)
*
* convMod<mod>(a, b) → c[k] = (Σ a[i]*b[k-i]) % mod
*
* Entradas devem estar em [0, mod).
* Seguro se N·log₂N·mod < 8.6·10¹⁴.
*
* Alta precisão sem mod: troque cut = 1<15 e remova os % mod.
*/

3F9 template<int mod>

```

```

11F vector<long long> convMod(
87D     const vector<long long>& a, const vector<long long>& b) {
F88     if (a.empty() || b.empty()) return {};
CE0     vector<long long> res(a.size() + b.size() - 1);
C69     int B = 32 - __builtin_clz(res.size()), n = 1 << B;
528     int cut = (int)sqrt(mod); // alta precisao: use l<=15 e tire os % mod abaixo
584     vector<CD> L(n), R(n), outs(n), outl(n);
5B0     for (int i = 0; i < (int)a.size(); i++)
8B7         L[i] = CD((int)a[i] / cut, (int)a[i] % cut);
81B     for (int i = 0; i < (int)b.size(); i++)
F55         R[i] = CD((int)b[i] / cut, (int)b[i] % cut);
628     fft(L); fft(R);
6B3     for (int i = 0; i < n; i++) {
39D         int j = -i & (n-1);
65E         outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
482         outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / CD(0, 1);
1C6     }
41B     fft(outl); fft(outs);
6D1     for (int i = 0; i < (int)res.size(); i++) {
// alta precisao: tire os % mod nas linhas abaixo
852         long long av = (long long)(real(outl[i]) + .5) % mod;
8CD         long long bv = (long long)(imag(outl[i]) + .5)
792             + (long long)(real(outs[i]) + .5);
7AE         long long cv = (long long)(imag(outs[i]) + .5);
557         res[i] = ((av * cut + bv) % mod * cut + cv) % mod;
6CB     }
B50     return res;
B73 }

```

NTT

```

/* NTT — Number Theoretic Transform
* Complexidade: O(N log N)
*
* NTT<mod, g>::conv(a, b) → c[k] = (Σ a[i]*b[k-i]) % mod
*
* Entradas devem estar em [0, mod).
* mod deve ser primo da forma 2^a*b+1 (garante raiz de unidade de ordem 2^a).
* g e a raiz primitiva de mod — nao e escolha livre, e fixo para cada mod.
*
* Opcoes de {mod, g}:
* {998244353, 3} → 119·2^23+1,      mais comum, N ≤ 2^3
* {469762049, 3} → 7·2^26+1,      N ≤ 2^6
* {167772161, 3} → 5·2^25+1,      N ≤ 2^5
* {2013265921, 31} → 15·2^27+1,   N ≤ 2^7 (mod > 10^9)
* {2281701377, 3} → 17·2^27+1,   N ≤ 2^7 (mod > 10^9)
* {7340033, 5} → 7·2^20+1,       N ≤ 2^0 (menor mod)
* {9223372036737335297LL, 3} → 549755813881·2^24+1, N≤2^4 (mod=2^9, ll)
*/
E34 template<ll mod, ll g>
F12 struct NTT {
D38     static ll fexp(ll a, ll b) {
C8D         ll res = 1; a %= mod;
FF0         for (; b > 0; b >= 1, a = a*a%mod)
602             if (b & 1) res = res*a%mod;
B50         return res;
B0B     }
936     static void ntt(vector<ll>& a) {
A5B         int n = a.size(), L = 31 - __builtin_clz(n);
D51         static vector<ll> rt(2, 1);
8EE         for (static int k = 2, s = 2; k < n; k *= 2, s++) {
335             rt.resize(n);
277             ll z[] = {1, fexp(g, mod >> s)};
631             for (int i = k; i < 2*k; i++) rt[i] = rt[i/2] * z[i&1] % mod;
D00         }
808         vector<int> rev(n);
5EB         for (int i = 0; i < n; i++) rev[i] = (rev[i/2] | (i&1) << L) / 2;
EE4         for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
657         for (int k = 1; k < n; k *= 2)
1E5             for (int i = 0; i < n; i += 2*k) {
OC2                 for (int j = 0; j < k; j++) {
86E                     ll z = rt[j+k] * a[i+j+k] % mod, &ai = a[i+j];
598                     a[i+j+k] = ai - z + (z > ai ? mod : 0);
488                     ai += z - (ai+z >= mod ? mod : 0);
D6A                 }
C0C             }
46D         static vector<ll> conv(vector<ll>& a, vector<ll>& b) {
F88             if (a.empty() || b.empty()) return {};
375             int s = a.size()+b.size()-1, n = 1 << (32 - __builtin_clz(s));
A16             a.resize(n); b.resize(n);
875             ntt(a); ntt(b);
DDC             ll inv = fexp(n, mod-2);
6F8             vector<ll> out(n); // separado: escrever em b corromperia b[n-i]
7AF             for (int i = 0; i < n; i++) out[-i&(n-1)] = a[i]*b[i]%mod*inv%mod;
EC9             ntt(out);
C20             return {out.begin(), out.begin()+s};
0CF         }
EFD };

```

FWHT

```

/* FWHT — Fast Walsh-Hadamard Transform
* Complexidade: O(N log N)
*
* Convolucao bitwise: c[k] = Σ a[i]*b[j] onde i op j = k
*
* fwht_conv<'^'>(a, b) → convolucao XOR
* fwht_conv<'>'>(a, b) → convolucao OR
* fwht_conv<'&'>(a, b) → convolucao AND
*
* Tamanhos sao ajustados para a proxima potencia de 2 automaticamente.
*/
597 template<char op>
823 void fwht(vector<ll>& a, bool inv = false) {
94D     int n = a.size();
980     for (int len = 1; len < n; len *= 2)
EBC         for (int i = 0; i < n; i += 2*len)
7AB             for (int j = 0; j < len; j++) {
032                 ll u = a[i+j], v = a[i+j+len];
FBD                 if (op == '^') a[i+j] = u+v, a[i+j+len] = u-v;
02F                 if (op == '|') a[i+j+len] = v + (inv ? -u : +u);
729                 if (op == '&') a[i+j] = u + (inv ? -v : +v);
1C4             }
163         if (op == '^' && inv) for (auto& x : a) x /= n;
643 }
597 template<char op>
E0F vector<ll> fwht_conv(vector<ll> a, vector<ll> b) {
43E     int n = 1;
3B9     while (n < (int)max(a.size(), b.size())) n *= 2;
A16     a.resize(n); b.resize(n);
357     fwht<op>(a); fwht<op>(b);
34E     for (int i = 0; i < n; i++) a[i] *= b[i];
9D7     fwht<op>(a, true);
3F5     return a;
22D }

```

String

KMP

```

/* KMP (Knuth-Morris-Pratt)
* failure[i] = tamanho do maior prefixo proprio de s[0..i] que e tambem sufixo.
* Complexidade: O(N) build, O(N + M) busca.
*
* Aplicacoes:
* - Busca de padrao P em texto T em O(|P| + |T|).
* - Menor periodo da string: N - failure[N-1].
* - Contar ocorrencias de todos os prefixos como sufixos.
*/
090 vector<int> kmp_failure(const string& s) {
163     int n = s.size();
D11     vector<int> f(n, 0);
6F5     for (int i = 1; i < n; i++) {
844         int j = f[i - 1];
424         while (j > 0 && s[i] != s[j]) j = f[j - 1];
04B         if (s[i] == s[j]) j++;
199         f[i] = j;
D74     }
ABE     return f;
8B7 }
// Busca todas as ocorrencias de pattern em text. Retorna indices (0-indexed).
751 vector<int> kmp_search(const string& pattern, const string& text) {
365     string s = pattern + "$" + text;
3EC     vector<int> f = kmp_failure(s);
E6D     int p = pattern.size();
02F     vector<int> res;
D6B     for (int i = p + 1; i < (int)s.size(); i++)
31E         if (f[i] == p) res.push_back(i - 2 * p);
B50     return res;
2C2 }

```

Z Function

```

/* Z-Function
* z[i] = tamanho do maior prefixo de s que coincide com s[i..].
* z[0] = 0 por convencao.
* Complexidade: O(N) build.
*
* Aplicacoes:
* - Busca de padrao P em texto T: z-function de P + "$" + T.
* - Posicoes i onde z[i] == |P| sao ocorrencias de P em T.
* - Contar ocorrencias de prefixo de tamanho k: contar z[i] >= k.
* - Menor periodo da string: menor p tal que p divide N e z[p] == N - p.
*/
7AD vector<int> z_function(const string& s) {
163     int n = s.size();

```

```

2B1     vector<int> z(n, 0);
A5C     for (int i = 1, l = 0, r = 0; i < n; i++) {
20E         if (i < r) z[i] = min(r - i, z[i - l]);
4ED         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
B76         if (i + z[i] > r) l = i, r = i + z[i];
38B     }
070     return z;
197 }
// Busca todas as ocorrencias de pattern em text. Retorna indices (0-indexed).
430 vector<int> z_search(const string& pattern, const string& text) {
365     string s = pattern + "$" + text;
EA6     vector<int> z = z_function(s);
E6D     int p = pattern.size();
02F     vector<int> res;
D6B     for (int i = p + 1; i < (int)s.size(); i++)
3F4         if (z[i] == p) res.push_back(i - p - 1);
B50     return res;
0FE }

```

String Hash

```

/* String Hashing (Single e Double Hash)
* Hash polinomial para comparacao de substrings em O(1) apos build O(N).
* Complexidade: O(N) build, O(1) query.
* Memoria: O(N)
*
* SingleHash: um par (MOD, BASE) — risco baixo de colisao para uso geral.
* DoubleHash: dois pares independentes — colisao praticamente impossivel.
*
* Uso:
* SingleHash h(s);
* h.get(l, r); // hash de s[l..r]
* h.get(l, r) == h.get(l2, r2); // compara substrings
*/

```

```

/* --- Single Hash --- */
55A struct SingleHash {
029     static const ll MOD = (1LL << 61) - 1;
56C     static const ll BASE;
F42     vector<ll> h, pw;
87D     static ll mul(ll a, ll b) {
3A1         return (__uint128_t)a * b % MOD;
30E     }
67F     static ll add(ll a, ll b) {
9F4         a += b;
BF2         return a >= MOD ? a - MOD : a;
4E8     }
82B     SingleHash() {}
38D     SingleHash(const string& s) : h(s.size() + 1, 0), pw(s.size() + 1, 1) {
C46         for (int i = 0; i < (int)s.size(); i++) {
4DB             h[i + 1] = add(mul(h[i], BASE), s[i]);
F0F             pw[i + 1] = mul(pw[i], BASE);
0FA         }
231     }
BA8     ll get(int l, int r) const {
128         return add(h[r + 1], MOD - mul(h[l], pw[r - l + 1]));
358     }
4EF };
E2C const ll SingleHash::BASE = []() {
798     mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
6FB     return uniform_int_distribution<ll>(le9, SingleHash::MOD - 1)(rng);
7E5 }();

```

```

/* --- Double Hash --- */
2AF struct DoubleHash {
172     static const ll MOD1 = (1LL << 61) - 1;
D89     static const ll MOD2 = le9 + 9;
2BF     static const ll BASE1, BASE2;
23A     vector<ll> h1, h2, pw1, pw2;
583     static ll mul1(ll a, ll b) { return (__uint128_t)a * b % MOD1; }
577     static ll add1(ll a, ll b) { a += b; return a >= MOD1 ? a - MOD1 : a; }
997     static ll mul2(ll a, ll b) { return (__uint128_t)a * b % MOD2; }
AC6     static ll add2(ll a, ll b) { a += b; return a >= MOD2 ? a - MOD2 : a; }
A3E     DoubleHash() {}
3C9     DoubleHash(const string& s)
01A         : h1(s.size() + 1, 0), h2(s.size() + 1, 0),
4CF         pw1(s.size() + 1, 1), pw2(s.size() + 1, 1) {
C46         for (int i = 0; i < (int)s.size(); i++) {
9F0             h1[i + 1] = add1(mul1(h1[i], BASE1), s[i]);
128             h2[i + 1] = add2(mul2(h2[i], BASE2), s[i]);
562             pw1[i + 1] = mul1(pw1[i], BASE1);
961             pw2[i + 1] = mul2(pw2[i], BASE2);
706         }
366     }
// retorna par de hashes de s[l..r] (0-indexed, inclusive)
C89     pair<ll, ll> get(int l, int r) const {
21E         ll v1 = add1(h1[r + 1], MOD1 - mul1(h1[l], pw1[r - l + 1]));

```

```

CA9         ll v2 = add2(h2[r + 1], MOD2 - mul2(h2[l], pw2[r - l + 1]));
F8E         return {v1, v2};
F62     }
331 };

BBD const ll DoubleHash::BASE1 = []() {
798     mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
907     return uniform_int_distribution<ll>(1e9, DoubleHash::MOD1 - 1)(rng);
C70 }();

BBD const ll DoubleHash::BASE2 = []() {
A93     mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count() + 1);
AB4     return uniform_int_distribution<ll>(1e9, DoubleHash::MOD2 - 1)(rng);
13F }();

```

Trie

```

/* Trie (Prefix Tree)
 * Insercao e busca de strings em O(|s|).
 * Memoria: O(N * ALPHA) onde N e o total de caracteres inseridos.
 * Requisitos: strings sobre o alfabeto [base, base + ALPHA).
 */

```

```

71F template<int ALPHA = 26, char BASE = 'a'>
71A struct Trie {
BF2     struct Node {
C02         int ch[ALPHA];
0F9         int cnt;
302         int end;
D7F         Node() : cnt(0), end(0) { fill(ch, ch + ALPHA, -1); }
B69     };
F5B     vector<Node> t;
8E7     Trie() { t.emplace_back(); }
E7D     void insert(const string& s) {
580         int no = 0;
B4F         for (char c : s) {
FE1             int b = c - BASE;
F67             if (t[no].ch[b] == -1) {
06C                 t[no].ch[b] = t.size();
83D                 t.emplace_back();
6E6             }
C8F             no = t[no].ch[b];
429             t[no].cnt++;
098         }
0B3         t[no].end++;
1A2     }
BA9     bool search(const string& s) const {
580         int no = 0;
B4F         for (char c : s) {
FE1             int b = c - BASE;
1DD             if (t[no].ch[b] == -1) return false;
C8F             no = t[no].ch[b];
1D5         }
A49         return t[no].end > 0;
3BE     }
0B1     int count_prefix(const string& s) const {
580         int no = 0;
B4F         for (char c : s) {
FE1             int b = c - BASE;
FE4             if (t[no].ch[b] == -1) return 0;
C8F             no = t[no].ch[b];
A50         }
395         return t[no].cnt;
EF9     }
4E2     int count(const string& s) const {
580         int no = 0;
B4F         for (char c : s) {
FE1             int b = c - BASE;
FE4             if (t[no].ch[b] == -1) return 0;
C8F             no = t[no].ch[b];
A50         }
1D1         return t[no].end;
34B     }
E20 };

```

Manacher

```

/* Manacher's Algorithm (Palindrome Detection)
 * Encontra todos os palindromos em uma string em tempo linear.
 * Complexidade: O(N) onde N e o tamanho da string.
 * Memoria: O(N)
 * Funcoes:
 * - manacher(s): arr[i] = tamanho do maior palindromo centrado em i.
 * - palindrome: Struct para verificar se s[i..j] e palindromo em O(1).
 * - pal_begin(s): arr[i] = tamanho do maior palindromo começando em i.
 * - pal_end(s): arr[i] = tamanho do maior palindromo terminando em i.
 */

```

```

67A template<typename T>
44D vector<int> manacher(const T& s) {
18F     int l = 0, r = -1, n = s.size();

```

```

FC9     vector<int> d1(n), d2(n);
603     for (int i = 0; i < n; i++) {
821         int k = i > r ? 1 : min(d1[l + r - i], r - i);
61A         while (i + k < n && i - k >= 0 && s[i + k] == s[i - k]) k++;
61E         d1[i] = k--;
9F6         if (i + k > r) l = i - k, r = i + k;
950     }
E03     l = 0, r = -1;
603     for (int i = 0; i < n; i++) {
1A6         int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
AC1         k++;
A94         while (i + k <= n && i - k >= 0 && s[i + k - 1] == s[i - k])
AC1             k++;
EAA         d2[i] = --k;
26D         if (i + k - 1 > r) l = i - k, r = i + k - 1;
4FE     }
C41     vector<int> ret(2 * n - 1);
E6B     for (int i = 0; i < n; i++) ret[2 * i] = 2 * d1[i] - 1;
E1D     for (int i = 0; i < n - 1; i++) ret[2 * i + 1] = 2 * d2[i + 1];
EDF     return ret;
9CA }

```

```

67A template<typename T>
BF0 struct palindrome {
F97     vector<int> man;
B2D     palindrome(const T& s) : man(manacher(s)) {}
9D7     bool query(int i, int j) {
BAD         return man[i + j] >= j - i + 1;
1E7     }
933 };

```

```

67A template<typename T>
10D vector<int> pal_begin(const T& s) {
542     int n = s.size() - 1;
E57     vector<int> ret(s.size());
FDE     palindrome<T> p(s);
687     ret[n] = 1;
45B     for (int i = n - 1; i >= 0; i--) {
4AD         ret[i] = min(ret[i + 1] + 2, n - i + 1);
D0D         while (!p.query(i, i + ret[i] - 1)) ret[i]--;
A70     }
EDF     return ret;
4E1 }

```

```

67A template<typename T>
934 vector<int> pal_end(const T& s) {
E57     vector<int> ret(s.size());
FDE     palindrome<T> p(s);
D51     ret[0] = 1;
88E     for (int i = 1; i < s.size(); i++) {
A32         ret[i] = min(ret[i - 1] + 2, i + 1);
6EA         while (!p.query(i - ret[i] + 1, i)) ret[i]--;
78E     }
EDF     return ret;
2E4 }

```

Suffix Array (NlogN)

```

/* Suffix Array O(N log N) - Prefix Doubling com Count Sort
 * sa[i] = indice do i-esimo sufixo em ordem lexicografica.
 * rk[i] = rank do sufixo s[i..] (inverso de sa).
 * lcp[i] = LCP entre sa[i-1] e sa[i] (lcp[0] = 0).
 * Adiciona '$' ao final da string internamente.
 * Complexidade: O(N log N) build, O(|P| log N) search.
 *
 * Aplicacoes:
 * - Busca de padrao P: search(p) retorna [lo, hi] no SA.
 * - Substring mais longa repetida: *max_element(lcp).
 * - Numero de substrings distintas: N*(N+1)/2 - sum(lcp).
 * - LCS de duas strings: SuffixArray::lcs(s, t).
 */

```

```

3F4 struct SuffixArray {
AC0     string s;
58B     vector<int> sa, rk, lcp;
264     SuffixArray() {}
357     SuffixArray(string s) : s(s) {
6F2         build();
E8F         build_lcp();
A87     }
0A8     void build() {
FC8         s += '$';
163         int n = s.size();
F27         sa.resize(n); rk.resize(n);
413         vector<pair<char, int>> a(n);
490         for (int i = 0; i < n; i++) a[i] = {s[i], i};
3F8         sort(a.begin(), a.end());
67D         for (int i = 0; i < n; i++) sa[i] = a[i].second;
F27         rk[sa[0]] = 0;
AA4         for (int i = 1; i < n; i++)
C6A             rk[sa[i]] = rk[sa[i - 1]] + (a[i].first != a[i - 1].first ? 1 : 0);

```

```

8A4     for (int k = 0; (1 << k) < n; k++) {
8A3         for (int i = 0; i < n; i++) sa[i] = (sa[i] - (1 << k) + n) % n;
188         count_sort();
037         vector<int> new_rk(n);
CCE         new_rk[sa[0]] = 0;
6F5         for (int i = 1; i < n; i++) {
325             pair<int, int> prv = {rk[sa[i-1]], rk[(sa[i-1] + (1 << k)) % n]};
E09             pair<int, int> now = {rk[sa[i]], rk[(sa[i] + (1 << k)) % n]};
1E5             new_rk[sa[i]] = new_rk[sa[i - 1]] + (now != prv ? 1 : 0);
DF8         }
14E         rk = new_rk;
F2D     }
B01 }
00C     void count_sort() {
9A8         int n = sa.size();
7C4         vector<int> new_sa(n), cnt(n, 0), pos(n, 0);
CBF         for (auto e : rk) cnt[e]++;
3D7         for (int i = 1; i < n; i++) pos[i] = pos[i - 1] + cnt[i - 1];
1EC         for (auto e : sa) new_sa[pos[rk[e]]++] = e;
253         sa = new_sa;
A28     }
47E     void build_lcp() {
9A8         int n = sa.size();
75C         lcp.assign(n, 0);
617         for (int i = 0, h = 0; i < n; i++) {
595             if (rk[i] == 0) { h = 0; continue; }
0A2             int j = sa[rk[i] - 1];
28E             while (s[i + h] == s[j + h]) h++;
CA3             lcp[rk[i]] = h;
254             if (h) h--;
7DF         }
2BE     }

```

```

// Retorna [lo, hi] no SA onde p ocorre. hi - lo = n² de ocorrencias.
pair<int, int> search(const string& p) const {
    int lo, hi, n = sa.size();
    {
        int l = 0, r = n;
        while (l < r) {
            int m = (l + r) / 2;
            if (s.compare(sa[m], p.size(), p) < 0) l = m + 1; else r = m;
        }
        lo = l;
    }
    {
        int l = 0, r = n;
        while (l < r) {
            int m = (l + r) / 2;
            if (s.compare(sa[m], p.size(), p) <= 0) l = m + 1; else r = m;
        }
        hi = l;
    }
    return {lo, hi};
}

```

```

// LCS entre duas strings. sep deve ser != '$' e nao aparecer em a/b.
static string lcs(const string& a, const string& b, char sep = 1) {
    SuffixArray suf(a + sep + b);
    int m = a.size(), best = 0, pos = 0;
    for (int i = 1; i < (int)suf.sa.size(); i++) {
        bool ga = suf.sa[i] < m, gb = suf.sa[i - 1] < m;
        if (ga != gb && suf.lcp[i] > best)
            { best = suf.lcp[i]; pos = suf.sa[i]; }
    }
    return suf.s.substr(pos, best);
}

```

Suffix Array (Linear)

```

/* Suffix Array O(N) - SA-IS (Induced Sorting)
 * Complexidade: O(N) build. Use apenas se N for muito grande (> 10^6).
 * Mais complexo que o O(N log N) - prefira suffix-array-nlogn.hpp.
 *
 * Campos publicos apos construaao:
 * sa[i] = indice do i-esimo sufixo em ordem lexicografica
 * rk[i] = rank do sufixo s[i..] (inverso de sa)
 * lcp[i] = tamanho do LCP entre sa[i-1] e sa[i]; lcp[0] = 0
 *
 * Metodos:
 * search(p) -> [lo, hi]: intervalo no SA onde p ocorre; hi-lo = n² ocorrencias
 */

```

```

FC8 struct SuffixArrayLinear {
AC0     string s;
58B     vector<int> sa, rk, lcp;
CAE     SuffixArrayLinear() {}
B7A     SuffixArrayLinear(const string& s) : s(s + '$') {
C75         vector<int> t(this->s.begin(), this->s.end());
C9A         sa = sa_is(t, 256);
9A8         int n = sa.size();

```



```

9BE     rk.resize(n);
665     for (int i = 0; i < n; i++) rk[sa[i]] = i;
E8F     build_lcp();
B40 }

B04 pair<int,int> search(const string& p) const {
B63     int lo, hi, n = sa.size();
F95     {
993         int l = 0, r = n;
40C         while (l < r) {
EE4             int m = (l + r) / 2;
A3B             if (s.compare(sa[m], p.size(), p) < 0) l = m + 1; else r = m;
103         }
0A8         lo = l;
CBD         }
F95     {
993         int l = 0, r = n;
40C         while (l < r) {
EE4             int m = (l + r) / 2;
E5A             if (s.compare(sa[m], p.size(), p) <= 0) l = m + 1; else r = m;
D29         }
C1B         hi = l;
277     }
AAF     return {lo, hi};
EEF }

47E void build_lcp() {
9A8     int n = sa.size();
75C     lcp.assign(n, 0);
E74     for (int i = 0, h = 0; i < n - 1; i++) {
0A2         int j = sa[rk[i] - 1];
28E         while (s[i + h] == s[j + h]) h++;
CA3         lcp[rk[i]] = h;
254         if (h) h--;
09C     }
C83 }

67A template<typename T>
EE7 static vector<int> sa_is(const T& s, int sigma) {
163     int n = s.size();
979     if (n == 1) return {0};
A94     if (n == 2) return s[0] < s[1] ? vector<int>{0, 1} : vector<int>{1, 0};

81A     vector<bool> t(n);
3A1     t[n - 1] = true;
9C8     for (int i = n - 2; i >= 0; i--)
537         t[i] = s[i] < s[i + 1] || (s[i] == s[i + 1] && t[i + 1]);

0C4     auto is_lms = [&](int i) { return i > 0 && t[i] && !t[i - 1]; };

FC6     vector<int> bkt(sigma + 1, 0);
7E5     for (int c : s) bkt[c + 1]++;
CA7     for (int i = 1; i <= sigma; i++) bkt[i] += bkt[i - 1];

CB5     auto induced_sort = [&](const vector<int>& lms) {
386         vector<int> sa(n, -1);
CDA         vector<int> b = bkt;
5EB         for (int i = (int)lms.size() - 1; i >= 0; i--)
F9D             sa[--b[s[lms[i]] + 1]] = lms[i];
332         b = bkt;
69A         for (int i = 0; i < n; i++) if (sa[i] > 0 && !t[sa[i] - 1])
420             sa[b[s[sa[i] - 1]] + 1] = sa[i] - 1;
332         b = bkt;
452         for (int i = n - 1; i >= 0; i--) if (sa[i] > 0 && t[sa[i] - 1])
3ED             sa[--b[s[sa[i] - 1] + 1]] = sa[i] - 1;
DB7         return sa;
66E     };

021     vector<int> lms;
DC0     for (int i = 0; i < n; i++) if (is_lms(i)) lms.push_back(i);
EF0     vector<int> sa = induced_sort(lms);

E41     vector<int> rank_(n, -1);
8F1     int cls = 0;
975     for (int i = 0, prev = -1; i < n; i++) {
FAD         if (!is_lms(sa[i])) continue;
837         if (prev != -1) {
047             bool diff = false;
5B8             for (int d = 0; ; d++) {
F37                 if (s[prev+d] != s[sa[i]+d] || t[prev+d] != t[sa[i]+d]) {
28C                     diff = true; break;
305                 }
585                 if (d > 0 && (is_lms(prev+d) || is_lms(sa[i]+d))) break;
DA4             }
265             if (diff) cls++;
142         }
A6F         rank_[sa[i]] = cls;
C6D         prev = sa[i];
FFD     }

CDB     vector<int> lms2, rank2;
830     for (int i = 0; i < n; i++)
96C         if (rank_[i] != -1) {
9FC             lms2.push_back(i); rank2.push_back(rank_[i]);
839         }

8F6     vector<int> sa2 = sa_is(rank2, cls + 1); // cls+1 classes distintas (0...

```

```

D14     vector<int> sorted_lms(lms2.size());
F29     for (int i = 0; i < (int)sa2.size(); i++) sorted_lms[i] = lms2[sa2[i]];
E0A     return induced_sort(sorted_lms);
134     }
051 };

```

Geometry

Point

```

107 const ld eps = 1e-9;
6F1 bool eq(ll a, ll b) { return a == b; }
BFC bool eq(ld a, ld b) { return abs(a - b) <= eps; }

67A template<typename T>
F26 struct Point {
645     T x, y;
0FA     Point(T x = 0, T y = 0) : x(x), y(y) {}

8D2     bool operator<(Point o) const { return tie(x, y) < tie(o.x, o.y); }
511     bool operator==(Point o) const { return eq(x, o.x) && eq(y, o.y); }
9E5     bool operator!=(Point o) const { return !(*this == o); }

480     Point operator+(Point o) const { return {x + o.x, y + o.y}; }
229     Point operator-(Point o) const { return {x - o.x, y - o.y}; }
AE8     Point operator*(T k) const { return {x * k, y * k}; }
7C2     Point operator/(T k) const { return {x / k, y / k}; }

61A     T dot(Point o) const { return x * o.x + y * o.y; }
CB8     T cross(Point o) const { return x * o.y - y * o.x; }
// area com sinal do triangulo (this, a, b)
61E     T cross(Point a, Point b) const { return (a - *this).cross(b - *this); }
F68     T dist2() const { return x * x + y * y; }

AD2     ld len() const { return hypot((ld)x, (ld)y); }
978     Point<ld> norm() const { ld l = len(); return {x / l, y / l}; }
178     Point perp() const { return {-y, x}; } // rotacao 90° CCW
8CF     Point<ld> rotate(ld a) const { // a em radianos
97E         return {x * cos(a) - y * sin(a), x * sin(a) + y * cos(a)};
CB4     }

539     friend ostream& operator<<(ostream& os, Point p) {
9B9         return os << p.x << " " << p.y;
DB4     }

2F4 };

29C using Pt = Point<ll>;
BA9 using Ptd = Point<ld>;

// angulo entre os vetores p e q
67A template<typename T>
447 ld angle(Point<T> p, Point<T> q) { return atan2((ld)p.cross(q), (ld)p.dot(q)); }

// area com sinal do triangulo (p, q, r) - positivo = CCW
67A template<typename T>
B80 T sarea(Point<T> p, Point<T> q, Point<T> r) { return p.cross(q, r); }

// se p, q, r sao colineares
67A template<typename T>
EB4 bool is_collinear(Point<T> p, Point<T> q, Point<T> r) {
B84     return eq(sarea(p, q, r), (T)0);
0D1 }

```

Line

```

C97 #include "point.hpp"

72C struct Line {
2DF     Ptd p, q;
2BD     Line() {}
299     Line(Ptd p, Ptd q) : p(p), q(q) {}
F5B };

// se p pertence ao segmento r
7E1 bool isinseg(Ptd p, Line r) {
AAC     Ptd a = r.p - p, b = r.q - p;
B7E     return eq(a.cross(b), 0) && a.dot(b) < eps;
C3B }

// projecao do ponto p na reta r
15B Ptd proj(Ptd p, Line r) {
BEA     if (r.p == r.q) return r.p;
2FF     Ptd v = r.q - r.p, u = p - r.p;
5C6     return r.p + v * (u.dot(v) / v.dot(v));
2E8 }

// intersecao das retas r e s (retorna {INF,INF} se paralelas)
24B Ptd inter(Line r, Line s) {
CAD     Ptd u = r.q - r.p, v = s.q - s.p, w = s.p - r.p;
018     ld d = u.cross(v);
F2F     if (eq(d, 0)) return {le18, le18};
2C9     return r.p + u * (w.cross(v) / d);
15F }

// se o segmento r intersecta o segmento s
090 bool interseg(Line r, Line s) {
CBC     if (isinseg(r.p, s) || isinseg(r.q, s) ||

```

```

826         isinseg(s.p, r) || isinseg(s.q, r)) return true;
D02     auto ccw = [] (Ptd a, Ptd b, Ptd c) {
9B9         return (b - a).cross(c - a) > eps;
3ED     };
13C     return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) &&
413         ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
7D4 }

// distancia do ponto p a reta r
E49 ld disttoline(Ptd p, Line r) {
FF0     return abs((r.q - r.p).cross(p - r.p)) / (r.q - r.p).len();
2F5 }

// distancia do ponto p ao segmento r
A20 ld distoseg(Ptd p, Line r) {
5A8     if ((r.q - r.p).dot(p - r.p) < 0) return (p - r.p).len();
A6E     if ((r.p - r.q).dot(p - r.q) < 0) return (p - r.q).len();
A19     return disttoline(p, r);
9C0 }

// distancia entre dois segmentos
68D ld distseg(Line a, Line b) {
4DF     if (interseg(a, b)) return 0;
3C3     return min({distoseg(a.p, b), distoseg(a.q, b),
458         distoseg(b.p, a), distoseg(b.q, a)});
A44 }

```

Circle

```

5FC #include "line.hpp"

// centro da circunferencia que passa por a, b, c
880 Ptd getcenter(Ptd a, Ptd b, Ptd c) {
3FB     Ptd m1 = (a + b) / 2, m2 = (a + c) / 2;
1E3     return inter(Line(m1, m1 + (b - a).perp()),
9A7         Line(m2, m2 + (c - a).perp()));
E56 }

// intersecao da circunferencia (c, r) com a reta ab
E6C vector<Ptd> circ_line_inter(Ptd a, Ptd b, Ptd c, ld r) {
B2F     vector<Ptd> ret;
FAD     Ptd u = b - a, v = a - c;
F37     ld A = u.dot(u), B = v.dot(u), C = v.dot(v) - r * r;
1FA     ld D = B * B - A * C;
818     if (D < -eps) return ret;
8B6     ret.push_back(c + v + u * (-B + sqrt(D + eps)) / A);
387     if (D > eps)
359         ret.push_back(c + v + u * (-B - sqrt(D)) / A);
EDF     return ret;
DAE }

// intersecao das circunferencias (a, r) e (b, R)
24A vector<Ptd> circ_inter(Ptd a, Ptd b, ld r, ld R) {
B2F     vector<Ptd> ret;
8C7     ld d = (b - a).len();
1B3     if (d > r + R + eps || d + min(r, R) < max(r, R) - eps) return ret;
398     ld x = (d * d - R * R + r * r) / (2 * d);
A5C     ld y = sqrt(max(0.0, r * r - x * x));
223     Ptd v = (b - a) / d;
CA4     ret.push_back(a + v * x + v.perp() * y);
9DA     if (y > eps)
07B         ret.push_back(a + v * x - v.perp() * y);
EDF     return ret;
0A2 }

```

Polygon

```

5FC #include "line.hpp"

// 2x a area com sinal (>0 = CCW)
67A template<typename T>
CB6 T area2(const vector<Point<T>&& poly) {
D0C     T a = 0;
BC6     int n = poly.size();
830     for (int i = 0; i < n; i++)
F7A         a += poly[i].cross(poly[(i + 1) % n]);
3F5     return a;
1B9 }

// area do poligono
67A template<typename T>
402 ld polarea(const vector<Point<T>&& poly) {
323     return abs((ld)area2(poly)) / 2.0;
3C3 }

// 0 = fora, 1 = dentro, 2 = na borda
// 0(n), funciona para qualquer poligono simples (convexo ou concavo)
67A template<typename T>
EA9 int winding(const vector<Point<T>&& poly, Point<T> p) {
C24     int n = poly.size(), w = 0;
603     for (int i = 0; i < n; i++) {
D79         Point<T> a = poly[i] - p, b = poly[(i+1)%n] - p;
972         if (a.y <= 0 && b.y > 0) {
E31             T c = a.cross(b);
346             if (c > 0) w++;
EAB             else if (c == 0) return 2;

```



```
305 } else if (b.y <= 0 && a.y > 0) {
E31     T c = a.cross(b);
6EA     if (c < 0) w--;
EAB     else if (c == 0) return 2;
992 } else if (b.y == 0 && a.y == 0) {
09D     if (min(a.x, b.x) <= 0 && 0 <= max(a.x, b.x)) return 2;
88E }
C79 }
2EA return w != 0 ? 1 : 0;
E31 }
```

```
// ponto no interior do casco convexo (CCW, sem colineares), 0(log n)
67A template<typename T>
406 bool inside_hull(const vector<Point<T>&> hull, Point<T> p) {
ACE     int n = hull.size();
D3E     if (n == 1) return p == hull[0];
4AF     if (n == 2) return hull[0].cross(hull[1], p) == 0
14D         && (p - hull[0]).dot(hull[1] - hull[0]) >= 0
E8C         && (p - hull[1]).dot(hull[0] - hull[1]) >= 0;
D16     if (hull[0].cross(hull[1], p) <= 0 ||
F02     hull[0].cross(hull[n-1], p) >= 0) return false;
B9E     int lo = 1, hi = n - 1;
3D1     while (lo + 1 < hi) {
C86         int mid = (lo + hi) / 2;
353         if (hull[0].cross(hull[mid], p) > 0) lo = mid;
8C0         else hi = mid;
575     }
3B9     return hull[lo].cross(hull[lo+1], p) >= 0;
FF3 }
```

```
// corta o poligono com a reta r, mantendo pontos p tais que (r.p, r.q, p) e CCW
010 vector<Ptd> cut_polygon(vector<Ptd> poly, Line r) {
B2F     vector<Ptd> ret;
BC6     int n = poly.size();
2B6     auto ccw = [&](Ptd a, Ptd b, Ptd c) { return (b - a).cross(c - a) > eps; };
603     for (int i = 0; i < n; i++) {
420         if (ccw(r.p, r.q, poly[i])) ret.push_back(poly[i]);
C6F         if (n == 1) continue;
F59         Line s(poly[i], poly[(i + 1) % n]);
DEA         Ptd p = inter(r, s);
A3D         if (isinség(p, s)) ret.push_back(p);
6D0     }
8A1     ret.erase(unique(ret.begin(), ret.end(), ret.end()));
780     if (ret.size() > 1 && ret.back() == ret[0]) ret.pop_back();
EDF     return ret;
FFE }
```

```
// se dois poligonos se intersectam - 0(n*m)
B8C bool interpol(const vector<Ptd>& v1, const vector<Ptd>& v2) {
7D1     int n = v1.size(), m = v2.size();
377     for (auto& p : v1) if (winding(v2, p)) return true;
4C6     for (auto& p : v2) if (winding(v1, p)) return true;
830     for (int i = 0; i < n; i++)
A75         for (int j = 0; j < m; j++)
2B6             if (interseg(Line(v1[i], v1[(i+1)%n]),
F1C                 Line(v2[j], v2[(j+1)%m]))) return true;
D1F     return false;
8F8 }
```

```
// Pick: I = (area2 - boundary + 2) / 2, boundary = Σgcd(|dx|, |dy|) nas arestas
B9D long long lattice_points(long long area2, long long boundary) {
4EE     return (area2 - boundary + 2) / 2;
32D }
```

```
// distancia entre dois poligonos
CCD ld distpol(const vector<Ptd>& v1, const vector<Ptd>& v2) {
F6B     if (interpol(v1, v2)) return 0;
7D1     int n = v1.size(), m = v2.size();
D63     ld ret = 1e18;
830     for (int i = 0; i < n; i++)
A75         for (int j = 0; j < m; j++)
7DB             ret = min(ret, distseg(Line(v1[i], v1[(i+1)%n]),
D87                 Line(v2[j], v2[(j+1)%m])));
EDF     return ret;
B9D }
```

Convex Hull

```
046 #include "polygon.hpp"

// Casco convexo CCW, O(N log N). Colineares sao descartados.
67A template<typename T>
580 vector<Point<T>> convex_hull(vector<Point<T>> pts) {
CA0     int n = pts.size();
3C9     if (n < 2) return pts;
CD7     sort(pts.begin(), pts.end());
9C8     vector<Point<T>> h;
721     h.reserve(2 * n);
107     for (int phase = 0; phase < 2; phase++) {
5C9         int start = h.size();
D9F         for (auto& p : pts) {
C31             while ((int)h.size() >= start + 2) {
531                 auto a = h[h.size()-2], b = h.back();
9F6                 if ((b - a).cross(p - a) > 0) break;
```

```
BFB         h.pop_back();
0ED     }
A49     h.push_back(p);
37A     }
BFB     h.pop_back();
05C     reverse(pts.begin(), pts.end());
0D0     }
E30     if (h.size() == 2 && h[0] == h[1]) h.pop_back();
81C     return h;
5AD }
```

```
// Struct para operacoes O(log n) em casco convexo (CCW, sem colineares).
// Construir com convex_hull() antes.
```

```
6E2 struct ConvexPol {
1C4     vector<Ptd> pol;
C45     ConvexPol() {}
D90     ConvexPol(vector<Ptd> v) : pol(convex_hull(v)) {}

// se o ponto esta dentro do casco - 0(log n)
46F     bool is_inside(Ptd p) {
B1C         int n = pol.size();
03D         if (n == 0) return false;
E08         return inside_hull(pol, p);
66C     }
```

```
// ponto extremo: cmp(p, q) = true se p e mais extremo que q
int extreme(const function<bool(Ptd, Ptd)& cmp) {
BAA     int n = pol.size();
B1C     auto extr = [&](int i, bool& cur_dir) {
4A2         cur_dir = cmp(pol[(i+1)%n], pol[i]);
22A         return !cur_dir && !cmp(pol[(i+n-1)%n], pol[i]);
35F     };
DDB     bool last_dir, cur_dir;
63D     if (extr(0, last_dir)) return 0;
A0D     int l = 0, r = n;
EAD     while (l + 1 < r) {
EE4         int m = (l + r) / 2;
F29         if (extr(m, cur_dir)) return m;
44A         bool rel_dir = cmp(pol[m], pol[l]);
72A         if (!last_dir && cur_dir ||
D60             (last_dir == cur_dir && rel_dir == cur_dir)) {
8A6             l = m;
1F1             last_dir = cur_dir;
69C         } else r = m;
D0E     }
792     return l;
B61 }
```

```
// indice do ponto com maior produto interno com v - 0(log n)
82A int max_dot(Ptd v) {
E52     return extreme([&](Ptd p, Ptd q) { return p.dot(v) > q.dot(v); });
C6D }
```

```
// tangentes a partir de p (indices {esquerda, direita}) - 0(log n)
D16 pair<int,int> tangents(Ptd p) {
1AF     auto L = [&](Ptd q, Ptd r) {
540         return (r - p).cross(q - p) > eps;
739     };
B2F     auto R = [&](Ptd q, Ptd r) {
F28         return (q - p).cross(r - p) > eps;
2E2     };
FA8     return {extreme(L), extreme(R)};
00E }
C9B ;
```

Closest Pair

```
C97 #include "point.hpp"

/* Closest Pair - Par de pontos mais proximos
* Complexidade: O(N log N)

* closest_pair(pts) - {i, j} indices do par mais proximo (i < j)
*
* Distancia ao quadrado: (pts[i] - pts[j]).dist2()
* Funciona com coordenadas inteiras (sem precisao flutuante).
*/
```

```
67A template<typename T>
CCC pair<int,int> closest_pair(const vector<Point<T>>& pts) {
CA0     int n = pts.size();
135     vector<int> idx(n);
295     iota(idx.begin(), idx.end(), 0);
658     sort(idx.begin(), idx.end(),
866         [&](int a, int b) { return pts[a].x < pts[b].x; });

3C9     T best = (pts[idx[0]] - pts[idx[1]]).dist2();
D94     pair<int,int> ans = {idx[0], idx[1]};

// set ordenado por (y, indice)
3B1 set<pair<T,int>> active;
637     for (int l = 0, r = 0; r < n; r++) {
FAF         T delta = (T)ceil(sqrt((double)best)) + 1;
E85         while (pts[idx[r]].x - pts[idx[l]].x > delta) {
9CB             active.erase({pts[idx[l]].y, idx[l]});
63B             l++;
```

```
34B         delta = (T)ceil(sqrt((double)best)) + 1;
563     }
98F     auto lo = active.lower_bound({pts[idx[r]].y - delta, INT_MIN});
388     auto hi = active.upper_bound({pts[idx[r]].y + delta, INT_MAX});
DC0     for (auto it = lo; it != hi; it++) {
E0A         T d = (pts[idx[r]] - pts[*it->second]).dist2();
C22         if (d < best) { best = d; ans = {idx[r], it->second}; }
A2C     }
0C6     active.insert({pts[idx[r]].y, idx[r]});
F7D }
591     if (ans.first > ans.second) swap(ans.first, ans.second);
BA7     return ans;
844 }
```

Others

Stress Test

```
P=code #codigo a testar
Q=brute #brute correto
C= #checker (vazio=cmp, ou ex: C=checker)

g++ ${P}.cpp -o sol -O2 || exit 1
g++ ${Q}.cpp -o brt -O2 || exit 1
g++ gen.cpp -o gen -O2 || exit 1
[ -n "$C" ] && { g++ ${C}.cpp -o chk -O2 || exit 1; }
for ((i = 1; ; i++)) do
    echo $i
    ./gen $i > in
    ./sol < in > out
    ./brt < in > out2
[ -n "$C" ] && ./chk in out out2 || cmp -s out out2
if [ $? -ne 0 ]; then
    echo "--> entrada:"
    cat in
    echo "--> saida code:"
    cat out
    echo "--> saida brute:"
    cat out2
    break
fi
done
```

Gen

```
42A #define endl '\n'

C8A mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
// int x = rng();

463 int uniform(int l, int r) {
A7F     uniform_int_distribution<int> uid(l, r);
F54     return uid(rng);
D9E }

E8D int main() {
886     ios::sync_with_stdio(false);
B95     cin.tie(0);
BB3     return 0;
268 }
```

Checker

```
F3B int main(int argc, char* argv[]) {
4DE     ifstream in(argv[1]), out_sol(argv[2]), out_brt(argv[3]);
BB3     return 0; // return 0 for AC, 1 for WA
231 }
```

Hash Function

0 hash de cada subseção é calculado com MD5 (3 chars) sobre o código sem comentários e sem espaços em branco. Usado para verificar integridade durante contest.

0 hash de uma linha com } é o hash de todo o código desde a { que a abriu.

Para verificar: copie o código, rode o verificador e compare o prefixo.

Hash Verifier

```
DE3 string getHash(string s){
E7F     ofstream("z.cpp") << s;
```

```
FC0  system("g++ -E -P -dD -fpreprocessed ./z.cpp"
041      " | tr -d '[:space:]' | md5sum > sh");
F24  ifstream("sh") >> s;
A15  return s.substr(0, 3);
84E }
E8D int main(){
973     string l, t;
C3E     stack<string> st({""});
C61     while(getline(cin, l)){
54F         t = l;
242         for(auto c : l)
A53             if(c == '{') st.push(""); else
733             if(c == '}') t = st.top()+l, st.pop();
189     cout << getHash(t) + " " + l << endl;
DB4     st.top() += t;
C14 }
692 }
```

Reference Tables

Types, Constants & Complexity

type	bits	max	fmt	valor	obs										
int	32	$\approx 2.1 \times 10^9$	%d	12! = 479001600	int										
uint	32	$\approx 4.3 \times 10^9$	%u	20! $\approx 2.4 \times 10^{18}$	ll										
ll	64	$\approx 9.2 \times 10^{18}$	%lld	$\binom{33}{16} \approx 1.2 \times 10^9$	int										
ull	64	$\approx 1.8 \times 10^{19}$	%llu	$\binom{66}{33} \approx 7.2 \times 10^{18}$	ll										
double	64	$10^{308}, \varepsilon \approx 10^{-15}$	%lf	<table><tr><th colspan="2">$2^{-1} \bmod p$</th></tr><tr><td>$10^9 + 7$</td><td>500000004</td></tr><tr><td>$10^9 + 9$</td><td>500000005</td></tr><tr><td>998244353</td><td>499122177</td></tr><tr><td>$2^{61} - 1$</td><td>1152921504606846976</td></tr></table>		$2^{-1} \bmod p$		$10^9 + 7$	500000004	$10^9 + 9$	500000005	998244353	499122177	$2^{61} - 1$	1152921504606846976
$2^{-1} \bmod p$															
$10^9 + 7$	500000004														
$10^9 + 9$	500000005														
998244353	499122177														
$2^{61} - 1$	1152921504606846976														
long dbl	80	$10^{4932}, \varepsilon \approx 10^{-18}$	%Lf												

n	$\pi(n)$	n	O viavel
10^4	1229	10^8	$O(n)$
10^5	9592	10^7	$O(n \log n)$
10^6	78498	5×10^3	$O(n^2)$
10^7	664579	500	$O(n^3)$
10^8	5761455	22	$O(2^n n)$
10^9	50847534	11	$O(n!)$

Combinatória

Somatórios

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}, \quad \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}, \quad \sum_{i=1}^n i^3 = n^2 \frac{(n+1)^2}{4}$$
$$\sum_{i=1}^n i^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k n^{m+1-k}$$
$$\sum_{i=0}^n c^i = \frac{c^{n+1}-1}{c-1}, \quad \sum_{i=0}^{\infty} c^i = \frac{1}{1-c}, \quad |c| < 1$$
$$\sum_{i=0}^n i c^i = \frac{n c^{n+2} - (n+1) c^{n+1} + c}{(c-1)^2}, \quad \sum_{i=0}^{\infty} i c^i = \frac{c}{(1-c)^2}, \quad |c| < 1$$
$$H_n = \sum_{i=1}^n \frac{1}{i} \approx \ln n + 0.5772, \quad \ln n < H_n < \ln n + 1$$
$$\sum_{i=1}^n H_i = (n+1)H_n - n, \quad \sum_{i=1}^n i H_i = \frac{n(n+1)}{2} H_n - \frac{n(n-1)}{4}$$

Identidades Binomiais

$$\binom{n}{k} = \frac{n!}{(n-k)!k!} = \binom{n}{n-k} = \frac{n}{k} \binom{n-1}{k-1} = \binom{n-1}{k} + \binom{n-1}{k-1}$$
$$\binom{n}{m} \binom{m}{k} = \binom{n}{k} \binom{n-k}{m-k}, \quad \binom{n}{k} = (-1)^k \binom{k-n-1}{k}$$
$$(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k, \quad x^n - y^n = (x-y) \sum_{k=0}^{n-1} x^{n-1-k} y^k$$
$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad \sum_{k=0}^n (-1)^k \binom{n}{k} = 0$$
$$\sum_{k=0}^n k \binom{n}{k} = n 2^{n-1}, \quad \sum_{k=0}^n k^2 \binom{n}{k} = n(n+1) 2^{n-2}$$
$$\sum_{k=0}^n \binom{r+k}{k} = \binom{r+n+1}{n}, \quad \sum_{k=0}^n \binom{k}{m} = \binom{n+1}{m+1}$$
$$\sum_{k=0}^n \binom{r}{k} \binom{s}{n-k} = \binom{r+s}{n}, \quad \sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}$$
$$\sum_{k=0}^r (-1)^k \binom{n}{k} = (-1)^r \binom{n-1}{r}$$
$$(1+x)^n = \sum_{k=0}^n \binom{n}{k} x^k, \quad (1+x)^{-n} = \sum_{k=0}^{\infty} (-1)^k \binom{n+k-1}{k} x^k$$

Inversao binomial: $f(n) = \sum_k \binom{n}{k} g(k) \Leftrightarrow g(n) = \sum_k (-1)^{n-k} \binom{n}{k} f(k)$

Contagem

Permutacoes com repeticoes $n_1, ..., n_r$: $\frac{n!}{n_1! \cdots n_r!}$

Permutacoes circulares: $(n-1)!$

Stars & Bars: $x_1 + \cdots + x_k = n, \quad x_i \geq 0$: $\binom{n+k-1}{k-1}$

Inclusao-Exclusao: $|\bigcup_{i=1}^m A_i| = \sum |A_i| - \sum |A_i \cap A_j| + \cdots \pm |A_1 \cap \cdots \cap A_n|$

Sequências Especiais

Catalan: $C_n = \frac{1}{n+1} \binom{2n}{n}$; 1, 1, 2, 5, 14, 42, 132, 429, ...

Conta: arvores binarias com n nos, triangulacoes de $(n+2)$ -gono, n pares de parenteses validos.

Fibonacci: $F_0 = F_1 = 1, \quad F_{-i} = (-1)^{i-1} F_i$
 $F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}, \quad \varphi = \frac{1 + \sqrt{5}}{2}, \quad \psi = \frac{1 - \sqrt{5}}{2}$

$$F_{i+1}F_{i-1} - F_i^2 = (-1)^i \quad (\text{Cassini})$$
$$F_{n+k} = F_k F_{n+1} + F_{k-1} F_n, \quad F_{2n} = F_n F_{n+1} + F_{n-1} F_n$$
$$\gcd(F_m, F_n) = F_{\gcd(m,n)}, \quad \sum_{k=1}^n F_k = F_{n+2} - 1, \quad \sum_{k=1}^n F_k^2 = F_n F_{n+1}$$

$$\begin{pmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

Representacao de Fibonacci: todo n tem representacao unica $n = F_{k_1} + \cdots + F_{k_m}, \quad k_i \geq k_{i+1} + 2, \quad k_m \geq 2$.

Stirling 1ª especie $\begin{bmatrix} n \\ k \end{bmatrix}$: permutacoes de n com k ciclos.

$$\begin{bmatrix} n \\ 1 \end{bmatrix} = (n-1)!, \quad \begin{bmatrix} n \\ 2 \end{bmatrix} = (n-1)! H_{n-1}, \quad \begin{bmatrix} n \\ n \end{bmatrix} = 1$$
$$\begin{bmatrix} n \\ k \end{bmatrix} = (n-1) \begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix}, \quad \sum_k \begin{bmatrix} n \\ k \end{bmatrix} = n!$$
$$x^{\overline{n}} = \sum_{k=0}^n \begin{bmatrix} n \\ k \end{bmatrix} x^k, \quad \begin{bmatrix} n \\ n-1 \end{bmatrix} = \binom{n}{2}$$

Stirling 2ª especie $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$: particoes de n em k subconjuntos nao vazios.

$$\left\{ \begin{matrix} n \\ 1 \end{matrix} \right\} = \begin{bmatrix} n \\ n \end{bmatrix} = 1, \quad \left\{ \begin{matrix} n \\ 2 \end{matrix} \right\} = 2^{n-1} - 1, \quad \left\{ \begin{matrix} n \\ n-1 \end{matrix} \right\} = \begin{bmatrix} n \\ n-1 \end{bmatrix} = \binom{n}{2}$$
$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\} + \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

Bell $B_n = \sum_k \begin{Bmatrix} n \\ k \end{Bmatrix}$: # particoes de $\{1, \dots, n\}$ em subconjuntos nao vazios.
 $B_0 = 1, B_1 = 1, B_2 = 2, B_3 = 5, B_4 = 15, B_5 = 52, B_6 = 203$

Numeros de Euler $\left\langle \begin{matrix} n \\ k \end{matrix} \right\rangle$: permutacoes de n com k ascendentes.

$$\left\langle \begin{matrix} n \\ 0 \end{matrix} \right\rangle = \left\langle \begin{matrix} n \\ n-1 \end{matrix} \right\rangle = 1, \quad \left\langle \begin{matrix} n \\ k \end{matrix} \right\rangle = \left\langle \begin{matrix} n-1 \\ k \end{matrix} \right\rangle + \left\langle \begin{matrix} n-1 \\ k-1 \end{matrix} \right\rangle, \quad \left\langle \begin{matrix} n \\ k \end{matrix} \right\rangle = (k+1) \left\langle \begin{matrix} n-1 \\ k \end{matrix} \right\rangle + (n-k) \left\langle \begin{matrix} n-1 \\ k-1 \end{matrix} \right\rangle$$
$$\left\langle \begin{matrix} n \\ 1 \end{matrix} \right\rangle = 2^n - n - 1, \quad \left\langle \begin{matrix} n \\ m \end{matrix} \right\rangle = \sum_{k=0}^m (-1)^k \binom{n+1}{k} (m+1-k)^n$$
$$x^n = \sum_{k=0}^n \left\langle \begin{matrix} n \\ k \end{matrix} \right\rangle \binom{x+k}{n}$$

Derangements: $D_n = (n-1)(D_{n-1} + D_{n-2})$; $D_n \approx \frac{n!}{e}$

Josephus: $f(1, k) = 0, \quad f(n, k) = (f(n-1, k) + k) \bmod n$

Funções Geradoras

Serie ordinaria: $A(x) = \sum_{i \geq 0} a_i x^i$; exponencial: $A(x) = \sum_{i \geq 0} a_i \frac{x^i}{i!}$
 $\frac{1}{1-x} = \sum x^i, \quad e^x = \sum \frac{x^i}{i!}, \quad \ln(1+x) = \sum (-1)^{i+1} \frac{x^i}{i}$
 $(1+x)^n = \sum \binom{n}{i} x^i, \quad \frac{x}{1-x-x^2} = \sum F_i x^i$
 $\frac{1}{2x} (1 - \sqrt{1-4x}) = \sum C_i x^i, \quad \frac{1}{\sqrt{1-4x}} = \sum \binom{2i}{i} x^i$

Convolucao: $A(x)B(x) = \sum_i \left(\sum_{j=0}^i a_j b_{i-j} \right) x^i$.

Soma acumulada: $B(x) = \frac{1}{1-x} A(x)$ se $b_i = \sum_{j \leq i} a_j$.

Potências Fatoriais & Bernoulli

$$x^{\underline{n}} = x(x-1)\cdots(x-n+1), \quad x^{\overline{n}} = x(x+1)\cdots(x+n-1)$$
$$x^{\underline{n}} = \sum_{k=1}^n \begin{bmatrix} n \\ k \end{bmatrix} (-1)^{n-k} x^k, \quad x^{\overline{n}} = \sum_{k=1}^n \begin{bmatrix} n \\ k \end{bmatrix} x^k$$

$$x^n = \sum_{k=1}^n \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^{\underline{k}}$$

Numeros de Bernoulli ($B_i = 0$ para i impar > 1):

$$B_0 = 1, \quad B_1 = -\frac{1}{2}, \quad B_2 = \frac{1}{6}, \quad B_4 = -\frac{1}{30}, \quad B_6 = \frac{1}{42}, \quad B_8 = -\frac{1}{30}$$
$$\frac{x}{e^x - 1} = \sum_{i \geq 0} B_i \frac{x^i}{i!}, \quad n! = \sqrt{2\pi n} \left(\frac{n}{e} \right)^n \left(1 + \Theta\left(\frac{1}{n}\right) \right)$$

Teoria dos Números

Euler Phi & Möbius

$\varphi(n)$: quantidade de inteiros em $[1, n]$ coprimos com n .

$$\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p} \right) = \prod_{i=1}^n p_i^{e_i-1} (p_i - 1)$$

$$\varphi(mn) = \varphi(m)\varphi(n) \quad (\gcd = 1), \quad \sum_{d \mid n} \varphi(d) = n$$

$$a^{\varphi(m)} \equiv 1 \pmod{m}$$

$$\mu(n) = \begin{cases} 1 & n = 1 \\ (-1)^r n = p_1 \cdots p_r, & \sum_{d \mid n} \mu(d) = [n = 1] \\ 0 & p^2 \mid n \end{cases}$$

Inversão de Möbius: $g(n) = \sum_{d \mid n} f(d) \Leftrightarrow f(n) = \sum_{d \mid n} \mu\left(\frac{n}{d}\right) g(d)$

$$\frac{1}{\zeta(x)} = \sum_{i=1}^{\infty} \frac{\mu(i)}{i^x}, \quad \frac{\zeta(x-1)}{\zeta(x)} = \sum_{i=1}^{\infty} \frac{\varphi(i)}{i^x}, \quad \zeta^2(x) = \sum_{i=1}^{\infty} \frac{d(i)}{i^x}$$

GCD, Diofantinas & TCR

$\gcd(a, b) = \gcd(b, a \bmod b)$; $\text{lcm}(a, b) = ab / \gcd(a, b)$

Bezout: $ax + by = \gcd(a, b)$ sempre tem solução inteira. Em geral, $ax + by = c$ tem solução $\Leftrightarrow \gcd(a, b) \mid c$.

TCR: Sistema $x \equiv a_i \pmod{m_i}$ com m_i coprimos dois a dois tem solucao unica $\bmod M = \prod m_i$:

$$x = \sum_i a_i M_i (M_i^{-1} \bmod m_i), \quad M_i = \frac{M}{m_i}$$

Aritmetica Modular

Inverso modular: $a^{-1} \bmod m$ via $a^{m-2} \bmod m$ (requer m primo). Para todos os inversos $1 \dots n$ de uma vez:

$$\text{inv}[1] = 1, \quad \text{inv}[i] = -(m/i) \cdot \text{inv}[m \bmod i] \bmod m$$

Lucas: Para p primo, $\binom{n}{k} \bmod p = \binom{n \bmod p}{k \bmod p} \cdot \left(\left\lfloor \frac{n}{p} \right\rfloor \right) \bmod p$

Recorrências & Séries

Teorema Mestre

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad a \geq 1, \quad b > 1:$$
$$f(n) = O(n^{\log_b a - \epsilon}) \Rightarrow T(n) = \Theta(n^{\log_b a})$$
$$f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \log n)$$
$$f(n) = \Omega(n^{\log_b a + \epsilon}) \text{ e } af\left(\frac{n}{b}\right) \leq cf(n) \Rightarrow T(n) = \Theta(f(n))$$

Séries de Taylor

$$e^x = \sum \frac{x^i}{i!}, \quad \ln(1+x) = \sum (-1)^{i+1} \frac{x^i}{i}, \quad \ln \frac{1}{1-x} = \sum \frac{x^i}{i}$$
$$\sin x = \sum (-1)^i \frac{x^{2i+1}}{(2i+1)!}, \quad \cos x = \sum (-1)^i \frac{x^{2i}}{(2i)!}$$
$$\arctan x = \sum (-1)^i \frac{x^{2i+1}}{2i+1}, \quad \frac{1}{(1-x)^{n+1}} = \sum \binom{i+n}{i} x^i$$

Teoria dos Grafos

Teoremas

Euler: Circuito Euleriano $\Leftrightarrow$ todos os vertices tem grau par. Caminho Euleriano $\Leftrightarrow$ exatamente 2 vertices de grau impar.

Fórmula de Euler (planar): $V - E + F = 2$. Grafo planar simples: $E \leq 3V - 6$; planar bipartido: $E \leq 2V - 4$. Todo planar tem algum vertice de grau ≤ 5 .

Cayley: Número de árvores geradoras rotuladas de K_n e n^{n-2} .

Kirchhoff: Número de árvores geradoras = qualquer cofator da matriz Laplaciana $L = D - A$.

Konig: Em grafo bipartido: cobertura mínima de vértices = emparelhamento máximo; conjunto independente máximo = V - emparelhamento máximo.

Hall: Existe emparelhamento perfeito de $U \Leftrightarrow \forall S \subseteq U: |N(S)| \geq |S|$.

Menger: Número máximo de caminhos aresta-disjuntos entre s e t = tamanho do corte mínimo s - t .

Dilworth: Em poset, partição mínima em cadeias = anti-cadeia máxima.

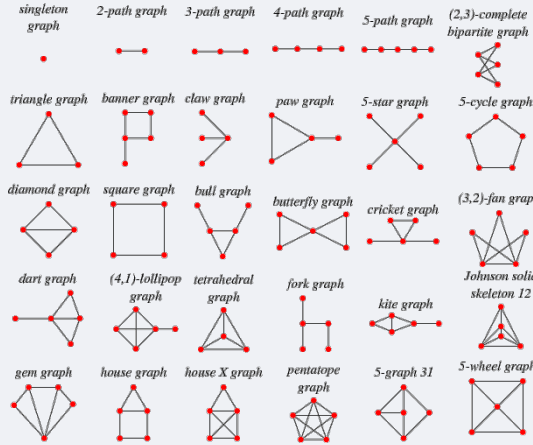
Mirsky: Em poset, partição mínima em anti-cadeias = comprimento da maior cadeia.

Prüfer: Bijecao entre árvores rotuladas de n vértices e sequências de comprimento $n-2$ sobre $\{1, \dots, n\}$.

Erdos-Gallai: Sequência $d_1 \geq \dots \geq d_n$ é sequência de graus de algum grafo simples $\Leftrightarrow \sum d_i$ e par e para todo k :

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k)$$

Tipos de Grafo



Matemática

Probabilidade

Bayes: Atualiza a probabilidade de A dado que B ocorreu.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Quando $P(B)$ nao e conhecida diretamente (A_j sao hipoteses mutuamente exclusivas e exaustivas):

$$P(A_i|B) = \frac{P(B|A_i)P(A_i)}{\sum_j P(B|A_j)P(A_j)}$$

Esperanca: $E[X] = \sum x P(X = x)$

Linearidade: $E[ax + bY] = aE[X] + bE[Y]$. Se independentes: $E[XY] = E[X]E[Y]$

Variancia: $\text{Var}(X) = E[X^2] - E[X]^2$; $\text{Var}(aX + b) = a^2 \text{Var}(X)$

Se independentes: $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$

Markov: $P[X \geq \lambda E[X]] \leq \frac{1}{\lambda}$

Chebyshev: $P[|X - E[X]| \geq \lambda \sigma] \leq \frac{1}{\lambda^2}$

Binomial $B(n, p)$: n ensaios de Bernoulli independentes com prob p .

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad E[X] = np, \quad \text{Var} = np(1-p)$$

Geometrica $\text{Geom}(p)$: numero de tentativas ate o primeiro sucesso ($q = 1 - p$).

$$P(X = k) = pq^{k-1}, \quad E[X] = \frac{1}{p}, \quad \text{Var} = \frac{q}{p^2}$$

Hipergeometrica $H(N, K, n)$: N itens, K marcados, n sorteados sem reposicao.

$$P(X = k) = \binom{K}{k} \frac{\binom{N-K}{n-k}}{\binom{N}{n}}, \quad E[X] = \frac{nK}{N}$$

Poisson

Pois(λ): aproximacao de $B(n,p)$ quando $n \rightarrow \infty, \quad np = \lambda.$

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}, \quad E[X] = \text{Var}(X) = \lambda$$

Normal

$\mathcal{N}(\mu, \sigma^2)$: $p(x) = \frac{1}{\sqrt{2\pi\sigma}} e^{-(x-\mu)^2/2\sigma^2}, \quad E[X] = \mu, \quad \text{Var} = \sigma^2.$

Coupon collector:

n tipos equiprovaveis; numero esperado de sorteios $= nH_n.$

Paradoxo do aniversario:

$\approx 1.18\sqrt{M}$ amostras de $[M]$ para colisao com prob $\geq 50\%.$

Trigonometria

$$\sin^2 x + \cos^2 x = 1, \quad 1 + \tan^2 x = \sec^2 x, \quad 1 + \cot^2 x = \csc^2 x$$
$$\sin(x \pm y) = \sin x \cos y \pm \cos x \sin y$$
$$\cos(x \pm y) = \cos x \cos y \mp \sin x \sin y$$
$$\tan(x \pm y) = \frac{\tan x \pm \tan y}{1 \mp \tan x \tan y}$$
$$\sin 2x = 2 \sin x \cos x, \quad \cos 2x = \cos^2 x - \sin^2 x = 2 \cos^2 x - 1 = 1 - 2 \sin^2 x$$
$$e^{ix} = \cos x + i \sin x, \quad e^{i\pi} = -1$$

Lei dos senos: $\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$

Lei dos cossenos: $c^2 = a^2 + b^2 - 2ab \cos C$

Area do triangulo (lados a,b,c; angulos A,B,C):
$$\frac{1}{2} ab \sin C, \quad \sqrt{s(s-a)(s-b)(s-c)} \quad s = \frac{a+b+c}{2}, \quad \frac{c^2 \sin A \sin B}{2 \sin C}$$

Circumraio: $R = ab \frac{c}{4A}.$ Inraio: $r = \frac{A}{s}.$

Geometria

Shoelace: $A = \frac{1}{2} |\sum_i (x_i y_{i+1} - x_{i+1} y_i)|$

Pick: $A = I + \frac{B^2}{2} - 1$ (I = pontos interiores, B = pontos na borda)

Area por coordenadas: $\frac{1}{2} |\det \begin{pmatrix} x_1-x_3 & x_2-x_3 \\ y_1-y_3 & y_2-y_3 \end{pmatrix}|$

Esferas/circulos: $A = \pi r^2, \quad V = \frac{4}{3} \pi r^3.$

Produto vetorial: $\vec{u} \times \vec{v} = (u_y v_z - u_z v_y, \quad u_z v_x - u_x v_z, \quad u_x v_y - u_y v_x)$

Em 2D: $u_x v_y - u_y v_x$ (>0 anti-horario, $=0$ colinear; modulo = area do paralelogramo)

Dist. ponto-ponto: $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ (3D: inclui $(z_1 - z_2)^2$)

Dist. ponto-reta ($ax + by + c = 0$): $d = |ax_0 + by_0 + c| \sqrt{a^2 + b^2}$

Rotacao 2D:
$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

Rotacao 3D em torno de $x, \quad y, \quad z$ por θ ($c = \cos \theta, \quad s = \sin \theta$):
$$R_x = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{pmatrix}, \quad R_y = \begin{pmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{pmatrix}, \quad R_z = \begin{pmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Teoria dos Jogos

Sprague-Grundy:

Todo jogo imparcial equivale a um Nim de um unico monte. O valor de Grundy (nimber) de uma posicao u e:
$$g(u) = \text{mex}\{g(v) : v \text{ alcançável de } u\}$$
onde $\text{mex}(S)$ e o menor inteiro nao negativo fora de $S.$

Posição perdedora $\Leftrightarrow g(u) = 0.$ Em jogo composto de partes independentes:
$$g = g_1 \oplus g_2 \oplus \dots \oplus g_k$$

Nim: k montes de tamanhos $a_1, \dots, a_k.$ Posição perdedora $\Leftrightarrow a_1 \oplus \dots \oplus a_k = 0.$

Nim misere (quem pegar o ultimo *perde*): subtraia 1 de cada pilha; se o XOR resultante $\neq 0,$ a posicao original e ganhadora.

Wythoff: 2 montes $(a,b), \quad a \leq b.$ Movimentos: remover qualquer qtd de um monte, ou a mesma qtd dos dois. Posição perdedora $\Leftrightarrow a = \lfloor \varphi k \rfloor, \quad b = \lfloor \varphi^2 k \rfloor$ para algum $k \geq 0,$ onde $\varphi = \frac{1+\sqrt{5}}{2}.$

Tabelas Numéricas

Números Primos

2	3	5	7	11	13	17	19	23	29	31	37	41	43	47	53	59	61	67	71	73
79	83	89	97	101	103	107	109	113	127	131	137	139	149	151	157	163	167	173	179	181
191	193	197	199	211	223	227	229	233	239	241	251	257	263	269	271	277	281	283	293	307
311	313	317	331	337	347	349	353	359	367	373	379	383	389	397	401	409	419	421	431	433
439	443	449	457	461	463	467	479	487	491	499	503	509	521	523	541	547	557	563	569	571
577	587	593	599	601	607	613	617	619	631	641	643	647	653	659	661	673	677	683	691	701
709	719	727	733	739	743	751	757	761	769	773	787	797	809	811	821	823	827	829	839	853
857	859	863	877	881	883	887	907	911	919	929	937	941	947	953	967	971	977	983	991	997

Fibonacci

n	F_n	n	F_n	n	F_n	n	F_n	n	F_n	n	F_n	n	F_n
0	0	1	1	2	1	3	2	4	3	5	5	6	8
7	13	8	21	9	34	10	55	11	89	12	144	13	233
14	377	15	610	16	987	17	1597	18	2584	19	4181	20	6765

Triângulo de Pascal ($\binom{n}{k}$)

$n \setminus k$	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	1	1									
2	1	2	1								
3	1	3	3	1							
4	1	4	6	4	1						
5	1	5	10	10	5	1					
6	1	6	15	20	15	6	1				
7	1	7	21	35	35	21	7	1			
8	1	8	28	56	70	56	28	8	1		
9	1	9	36	84	126	126	84	36	9	1	
10	1	10	45	120	210	252	210	120	45	10	1

Debugging

```
-g++ -O2 -std=c++17 -Wall -Wextra
-fsanitize=address,undefined
-fno-sanitize-recover=all -g
```

Antes de submeter:

- Escreva casos simples se o sample não for suficiente.
- Limite de tempo apertado? Gere casos máximos.
- Memória está ok?
- Algo pode dar overflow?
- Está submetendo o arquivo certo?

Wrong Answer:

- Imprima sua solução e outputs de debug.
- Está limpando estruturas entre casos de teste?
- O algoritmo cobre todo o range de entrada?
- Releia o enunciado completo.
- Todos os casos de borda estão corretos?
- Entendeu o problema corretamente?
- Variáveis não inicializadas?
- Overflow?
- Confundindo N/M, i/j, etc.?
- Tem certeza que o algoritmo funciona?
- Que casos especiais você não considerou?
- As funções da STL funcionam como você acha?
- Adicione asserts e resubmeta.
- Rode em casos criados manualmente.
- Execute o algoritmo num caso simples à mão.
- Explique o algoritmo para um colega.
- Peça para o colega revisar seu código.
- Dê uma volta, vá ao banheiro.
- O formato de saída está correto? (espaços, newlines)
- Reescreva do zero ou peça ao colega.

Runtime Error:

- Testou todos os casos de borda localmente?
- Variáveis não inicializadas?
- Lendo/escrivendo fora do range de algum vetor?
- Algum assert que pode falhar?
- Divisão por zero? (mod 0?)
- Recursão infinita possível?
- Ponteiros ou iteradores invalidados?
- Usando memória demais?

Time Limit Exceeded:

- Tem loop infinito?
- Qual é a complexidade do algoritmo?
- Está copiando dados desnecessariamente? (use referências)
- Tamanho da entrada/saída? (considere scanf)
- Evite map/vector; use arrays/unordered_map.
- O que seus colegas acham do algoritmo?

Memory Limit Exceeded:

- Quanto de memória seu algoritmo realmente precisa?
- Está limpando estruturas entre casos de teste?

17